# EP - Data Manipulation & Advanced Plotting (pandas, seaborn & 3D Plotting)

In this lab, we will study data manipulation and visualization using **pandas**, and explore the high level API of **seaborn** for generating visually appealing plots. We will also take a look at 3D plotting using **mplot3d**.

```
# Some IPython magic
# Put these at the top of every notebook, to get automatic reloading
and inline plotting
%reload_ext autoreload
%autoreload 2

import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

Check out these cheetsheets for fast reference to the common libraries:

**Cheat sheets:**

- python
- numpy
- matplotlib
- sklearn
- pandas

**Other:**

- Probabilities & Stats Refresher
- Algebra

## Pandas Crash Course

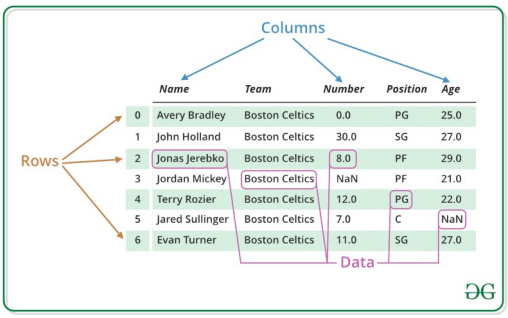
Pandas is a high-level data manipulation tool. It is built on the Numpy package and its key data structure is called the DataFrame. DataFrames allow you to store and manipulate tabular data in rows of observations and columns of variables.

Check this official guide for a started in pandas:

10 minutes to pandas

```
import pandas as pd
```

Pandas DataFrame is two-dimensional size-mutable, potentially heterogeneous tabular data structure with labeled axes (rows and columns). A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns. Pandas DataFrame consists of three principal components, the data, rows, and columns.




Let's load a publicly available *.csv* dataset into a pandas ***DataFrame***. We will use the popular *iris* dataset.

```
file_name = "https://raw.githubusercontent.com/uiuc-cse/data-fa14/gh-pages/data/iris.csv"

df = pd.read_csv(file_name)
df.head(n = 10)
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 150,\n  \"fields\": [\n    {\n      \"column\": \"sepal_length\",\n      \"properties\": {\n\"dtype\": \"number\",\n        \"std\": 0.8280661279778629,\n\"min\": 4.3,\n        \"max\": 7.9,\n        \"num_unique_values\": 35,\n        \"samples\": [\n          6.2,\n          4.5,\n          5.6\n        ],\n        \"semantic_type\": \"\",\n\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"sepal_width\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.4335943113621737,\n        \"min\": 2.0,\n        \"max\": 4.4,\n        \"num_unique_values\": 23,\n\"samples\": [\n          2.3,\n          4.0,\n          3.5\n],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n}\n    },\n    {\n      \"column\": \"petal_length\",\n\"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1.7644204199522617,\n        \"min\": 1.0,\n        \"max\": 6.9,\n\"num_unique_values\": 43,\n        \"samples\": [\n          6.7,\n3.8,\n          3.7\n        ],\n        \"semantic_type\": \"\",\n\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"petal_width\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.7631607417008414,\n        \"min\": 0.1,\n        \"max\": 2.5,\n        \"num_unique_values\": 22,\n\"samples\": [\n          0.2,\n          1.2,\n          1.3\n],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n}\n    },\n    {\n      \"column\": \"species\",\n\"properties\": {\n        \"dtype\": \"category\",\n\"num_unique_values\": 3,\n        \"samples\": [\n
```

```
\"setosa\",\n          \"versicolor\",\n          \"virginica\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\n}","type":"dataframe","variable_name":"df"}
```


A dataframe's **.describe()** method offers descriptive statistics which include those that summarize the central tendency, dispersion and shape of a dataset's distribution, excluding NaN values.

```
df.describe()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 8,\n  \"fields\": [\n
{\n      \"column\": \"sepal_length\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 51.24711349471842,\n
\"min\": 0.8280661279778629,\n        \"max\": 150.0,\n
\"num_unique_values\": 8,\n        \"samples\": [\n
5.843333333333334,\n          5.8,\n          150.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"sepal_width\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
52.08647211421483,\n        \"min\": 0.4335943113621737,\n
\"max\": 150.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          3.0540000000000003,\n          3.0,\n
150.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"petal_length\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 51.835227940958106,\n        \"min\":
1.0,\n        \"max\": 150.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          3.758666666666666,\n          4.35,\n
150.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"petal_width\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 52.63663424340991,\n        \"min\":
0.1,\n        \"max\": 150.0,\n        \"num_unique_values\": 8,\n
\"samples\": [\n          1.1986666666666668,\n          1.3,\n
150.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


Let's see some ways in which we can access the DataFrames' data. Each column of a pandas `DataFrame` is a pandas `Series`.

```
df['petal_width']
```

```
0      0.2
1      0.2
2      0.2
3      0.2
4      0.2
      ...
145    2.3
```

```
146    1.9
147    2.0
148    2.3
149    1.8
Name: petal_width, Length: 150, dtype: float64
```

We can do any vectorized operation on a `Series`. Moreover, a pandas `Series` allows us to do conditional selection of rows in a `DataFrame`.

```
setosas = df[df['species'] == 'setosa']

setosas.head() # only setosa species selected
```


```
{"summary":"{\n  \"name\": \"setosas\",\n  \"rows\": 50,\n
\"fields\": [\n    {\n      \"column\": \"sepal_length\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.3524896872134512,\n        \"min\": 4.3,\n        \"max\": 5.8,\n
\"num_unique_values\": 15,\n        \"samples\": [\n          5.8,\n
5.2,\n          5.1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"sepal_width\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.38102439795469095,\n        \"min\":
2.3,\n        \"max\": 4.4,\n        \"num_unique_values\": 16,\n
\"samples\": [\n          3.5,\n          3.0,\n          3.9\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"petal_length\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.1735111594364455,\n        \"min\": 1.0,\n        \"max\": 1.9,\n
\"num_unique_values\": 9,\n        \"samples\": [\n          1.0,\n
1.3,\n          1.1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"petal_width\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.10720950308167837,\n        \"min\":
0.1,\n        \"max\": 0.6,\n        \"num_unique_values\": 6,\n
\"samples\": [\n          0.2,\n          0.4,\n          0.6\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"species\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 1,\n        \"samples\": [\n
\"setosa\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"setosas"}
```


We can add a new column to a pandas `DataFrame`, simply by specifying its name and its contents.

**NB**: the data added to the new column must be the same length as the rest of the `DataFrame`.

```
df['sepal_area'] = df['sepal_length'] * df['sepal_width'] # adding new
columns
df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 150,\n  \"fields\": [\
n    {\n      \"column\": \"sepal_length\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.8280661279778629,\n
\"min\": 4.3,\n        \"max\": 7.9,\n        \"num_unique_values\":
35,\n        \"samples\": [\n          6.2,\n          4.5,\n
5.6\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"sepal_width\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.4335943113621737,\n        \"min\":
2.0,\n        \"max\": 4.4,\n        \"num_unique_values\": 23,\n
\"samples\": [\n          2.3,\n          4.0,\n          3.5\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"petal_length\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.7644204199522617,\n        \"min\": 1.0,\n        \"max\": 6.9,\n
\"num_unique_values\": 43,\n        \"samples\": [\n          6.7,\n
3.8,\n          3.7\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"petal_width\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.7631607417008414,\n        \"min\":
0.1,\n        \"max\": 2.5,\n        \"num_unique_values\": 22,\n
\"samples\": [\n          0.2,\n          1.2,\n          1.3\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"species\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n
\"setosa\",\n          \"versicolor\",\n          \"virginica\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"sepal_area\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
3.368692612125483,\n        \"min\": 10.0,\n        \"max\": 30.02,\n
\"num_unique_values\": 109,\n        \"samples\": [\n          12.75,\
n          19.980000000000004,\n          18.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"df"}
```


We can work with `Series` as we work with numpy arrays. We perform Min-Max normalization on the `petal_length` column.

```
# Min-Max Normalization
df['petal_length'] = (df['petal_length'] - df['petal_length'].min()) /
(df['petal_length'].max() - df['petal_length'].min())
df.head()
```

```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 150,\n  \"fields\": [\
n    {\n      \"column\": \"sepal_length\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.8280661279778629,\n
\"min\": 4.3,\n        \"max\": 7.9,\n        \"num_unique_values\":
35,\n        \"samples\": [\n          6.2,\n          4.5,\n
5.6\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"sepal_width\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.4335943113621737,\n        \"min\":
2.0,\n        \"max\": 4.4,\n        \"num_unique_values\": 23,\n
\"samples\": [\n          2.3,\n          4.0,\n          3.5\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"petal_length\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.29905430846648523,\n        \"min\": 0.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 43,\n        \"samples\": [\n
0.9661016949152542,\n          0.47457627118644063,\n
0.4576271186440678\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"petal_width\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.7631607417008414,\n        \"min\":
0.1,\n        \"max\": 2.5,\n        \"num_unique_values\": 22,\n
\"samples\": [\n          0.2,\n          1.2,\n          1.3\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"species\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n
\"setosa\",\n          \"versicolor\",\n          \"virginica\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"sepal_area\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
3.368692612125483,\n        \"min\": 10.0,\n        \"max\": 30.02,\n
\"num_unique_values\": 109,\n        \"samples\": [\n          12.75,\
n          19.980000000000004,\n          18.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"df"}
```


We can also use the `.apply()` method on either a `Series` or a `DataFrame` to modify its contents, or create a new column.

```
def capitalize(col):
  return col.capitalize()

df['species'] = df['species'].apply(capitalize)
df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 150,\n  \"fields\": [\
n    {\n      \"column\": \"sepal_length\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.8280661279778629,\n
```

```
\"min\": 4.3,\n        \"max\": 7.9,\n        \"num_unique_values\":
35,\n        \"samples\": [\n          6.2,\n          4.5,\n
5.6\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"sepal_width\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.4335943113621737,\n        \"min\":
2.0,\n        \"max\": 4.4,\n        \"num_unique_values\": 23,\n
\"samples\": [\n          2.3,\n          4.0,\n          3.5\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"petal_length\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.29905430846648523,\n        \"min\": 0.0,\n        \"max\": 1.0,\n
\"num_unique_values\": 43,\n        \"samples\": [\n
0.9661016949152542,\n          0.47457627118644063,\n
0.4576271186440678\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"petal_width\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.7631607417008414,\n        \"min\":
0.1,\n        \"max\": 2.5,\n        \"num_unique_values\": 22,\n
\"samples\": [\n          0.2,\n          1.2,\n          1.3\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"species\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n
\"Setosa\",\n          \"Versicolor\",\n          \"Virginica\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"sepal_area\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
3.368692612125483,\n        \"min\": 10.0,\n        \"max\": 30.02,\n
\"num_unique_values\": 109,\n        \"samples\": [\n          12.75,\
n          19.980000000000004,\n          18.0\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"df"}
```


A `DataFrame` also has a `groupby` method, that allows us to work on groupings of rows.

```
df.groupby('species').mean()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 3,\n  \"fields\": [\n
{\n      \"column\": \"species\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 3,\n
\"samples\": [\n          \"Setosa\",\n          \"Versicolor\",\n
\"Virginica\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"sepal_length\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.7950605846936022,\n        \"min\":
5.006,\n        \"max\": 6.587999999999999,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          5.006,\n
5.936,\n          6.587999999999999\n        ],\n
```

```
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"sepal_width\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.3313246142380612,\n        \"min\": 2.77,\n        \"max\": 3.418,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          3.418,\n
2.77,\n          2.9739999999999998\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"petal_length\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.35416986355869406,\n        \"min\": 0.07864406779661016,\n
\"max\": 0.7715254237288136,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.07864406779661016,\n
0.5525423728813559,\n          0.7715254237288136\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"petal_width\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.8977980470759185,\n        \"min\": 0.24400000000000002,\n
\"max\": 2.026,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.24400000000000002,\n
1.3259999999999998,\n          2.026\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"sepal_area\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.6618774002113794,\n        \"min\": 16.5262,\n        \"max\":
19.6846,\n        \"num_unique_values\": 3,\n        \"samples\": [\n
17.2088,\n          16.5262,\n          19.6846\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```


We can also iterate through each group. A group is another `DataFrame`.

```
for name, group in df.groupby('species'):
  print("Group:", name)
  print(group.head())
  print("-----")
```

```
Group: Setosa
   sepal_length  sepal_width  petal_length  petal_width species
sepal_area
0           5.1          3.5      0.067797          0.2  Setosa
17.85
1           4.9          3.0      0.067797          0.2  Setosa
14.70
2           4.7          3.2      0.050847          0.2  Setosa
15.04
3           4.6          3.1      0.084746          0.2  Setosa
14.26
4           5.0          3.6      0.067797          0.2  Setosa
18.00
```

```
-----
Group: Versicolor
    sepal_length  sepal_width  petal_length  petal_width
species  \
50           7.0          3.2      0.627119          1.4  Versicolor

51           6.4          3.2      0.593220          1.5  Versicolor

52           6.9          3.1      0.661017          1.5  Versicolor

53           5.5          2.3      0.508475          1.3  Versicolor

54           6.5          2.8      0.610169          1.5  Versicolor


    sepal_area
50       22.40
51       20.48
52       21.39
53       12.65
54       18.20
-----
Group: Virginica
     sepal_length  sepal_width  petal_length  petal_width
species  \
100          6.3          3.3      0.847458          2.5  Virginica

101          5.8          2.7      0.694915          1.9  Virginica

102          7.1          3.0      0.830508          2.1  Virginica

103          6.3          2.9      0.779661          1.8  Virginica

104          6.5          3.0      0.813559          2.2  Virginica


     sepal_area
100       20.79
101       15.66
102       21.30
103       18.27
104       19.50
-----
```

## Joins

Pandas allows for joining two or more `DataFrames` together using a common key. We can also do vertical or horizontal concatenation .




```
df1 = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'],
                    'B': ['B0', 'B1', 'B2', 'B3'],
                    'C': ['C0', 'C1', 'C2', 'C3'],
                    'D': ['D0', 'D1', 'D2', 'D3']})
df2 = pd.DataFrame({'A': ['A0', 'A1', 'A4', 'A5'],
                    'B': ['B4', 'B5', 'B6', 'B7'],
                    'C': ['C4', 'C5', 'C6', 'C7'],
                    'D': ['D4', 'D5', 'D6', 'D7']})
df3 = pd.DataFrame({'A': ['A8', 'A9', 'A10', 'A11'],
                    'B': ['B8', 'B9', 'B10', 'B11'],
                    'C': ['C8', 'C9', 'C10', 'C11'],
                    'D': ['D8', 'D9', 'D10', 'D11']})

pd.concat([df1, df2, df3]).reset_index(drop = True)
```


```
{"summary":"{\n  \"name\": \"pd\",\n  \"rows\": 12,\n  \"fields\": [\n
{\n      \"column\": \"A\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 10,\n
\"samples\": [\n          \"A10\",\n          \"A1\",\n
\"A5\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"B\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 12,\n        \"samples\": [\n
\"B10\",\n          \"B9\",\n          \"B0\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"C\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 12,\n
\"samples\": [\n          \"C10\",\n          \"C9\",\n
\"C0\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
```

\"D\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 12,\n        \"samples\": [\n
\"D10\",\n          \"D9\",\n          \"D0\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}

```
pd.merge(df1, df2, on = 'A', how = 'left')
```

{"summary":"{\n  \"name\": \"pd\",\n  \"rows\": 4,\n  \"fields\": [\n
{\n      \"column\": \"A\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 4,\n
\"samples\": [\n          \"A1\",\n          \"A3\",\n
\"A0\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"B_x\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 4,\n        \"samples\": [\n          \"B1\",\n
\"B3\",\n          \"B0\"\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"C_x\",\n      \"properties\": {\n        \"dtype\":
\"string\",\n        \"num_unique_values\": 4,\n        \"samples\":
[\n          \"C1\",\n          \"C3\",\n          \"C0\"\n        ],\
n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"D_x\",\n      \"properties\": {\
n        \"dtype\": \"string\",\n        \"num_unique_values\": 4,\n
\"samples\": [\n          \"D1\",\n          \"D3\",\n
\"D0\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"B_y\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n          \"B5\",\n
\"B4\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"C_y\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n          \"C5\",\n
\"C4\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"D_y\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n          \"D5\",\n
\"D4\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}

## Saving `DataFrames`

Pandas offers a multitude of methods for saving `DataFrames`.

```
df.to_csv('out.csv', index = False) # saves it locally, check out the
files in the right tab of colab

df.to_json()
```

```
{"type":"string"}
```

```
print(df.head().to_markdown())
```

```
|    |   sepal_length |   sepal_width |   petal_length |   petal_width
| species   |   sepal_area |
|---:|---------------:|--------------:|---------------:|--------------
:|:----------|-------------:|
|  0 |            5.1 |           3.5 |      0.0677966 |           0.2
| Setosa    |        17.85 |
|  1 |            4.9 |           3   |      0.0677966 |           0.2
| Setosa    |        14.7  |
|  2 |            4.7 |           3.2 |      0.0508475 |           0.2
| Setosa    |        15.04 |
|  3 |            4.6 |           3.1 |      0.0847458 |           0.2
| Setosa    |        14.26 |
|  4 |            5   |           3.6 |      0.0677966 |           0.2
| Setosa    |        18    |
```

```
print(df.head().to_latex())
```

```
\begin{tabular}{lrrrrlr}
\toprule
 & sepal_length & sepal_width & petal_length & petal_width & species &
sepal_area \\
\midrule
0 & 5.100000 & 3.500000 & 0.067797 & 0.200000 & Setosa & 17.850000 \\
1 & 4.900000 & 3.000000 & 0.067797 & 0.200000 & Setosa & 14.700000 \\
2 & 4.700000 & 3.200000 & 0.050847 & 0.200000 & Setosa & 15.040000 \\
3 & 4.600000 & 3.100000 & 0.084746 & 0.200000 & Setosa & 14.260000 \\
4 & 5.000000 & 3.600000 & 0.067797 & 0.200000 & Setosa & 18.000000 \\
\bottomrule
\end{tabular}
```

```
print(df.head(n = 3).to_html())
```

```
<table border="1" class="dataframe">
  <thead>
    <tr style="text-align: right;">
      <th></th>
      <th>sepal_length</th>
      <th>sepal_width</th>
      <th>petal_length</th>
      <th>petal_width</th>
      <th>species</th>
      <th>sepal_area</th>
    </tr>
  </thead>
  <tbody>
    <tr>
```

```
      <th>0</th>
      <td>5.1</td>
      <td>3.5</td>
      <td>0.067797</td>
      <td>0.2</td>
      <td>Setosa</td>
      <td>17.85</td>
    </tr>
    <tr>
      <th>1</th>
      <td>4.9</td>
      <td>3.0</td>
      <td>0.067797</td>
      <td>0.2</td>
      <td>Setosa</td>
      <td>14.70</td>
    </tr>
    <tr>
      <th>2</th>
      <td>4.7</td>
      <td>3.2</td>
      <td>0.050847</td>
      <td>0.2</td>
      <td>Setosa</td>
      <td>15.04</td>
    </tr>
  </tbody>
</table>

df.to_sql(name = '<table_name>', con = '<connection>') # insert into a
sql database, works with a valid connection
# check out
https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Data
Frame.to_sql.html

---------------------------------------------------------------------
-----
ArgumentError                             Traceback (most recent call
last)
<ipython-input-35-c756941cb00c> in <cell line: 0>()
----> 1 df.to_sql(name = '<table_name>', con = '<connection>') #
insert into a sql database, works with a valid connection
      2 # check out
https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Data
Frame.to_sql.html

/usr/local/lib/python3.11/dist-packages/pandas/util/_decorators.py in
wrapper(*args, **kwargs)
    331                     stacklevel=find_stack_level(),
    332                 )
```

```
--> 333             return func(*args, **kwargs)
    334
    335         # error: "Callable[[VarArg(Any), KwArg(Any)], Any]"
has no

/usr/local/lib/python3.11/dist-packages/pandas/core/generic.py in
to_sql(self, name, con, schema, if_exists, index, index_label,
chunksize, dtype, method)
   3085         from pandas.io import sql
   3086
-> 3087         return sql.to_sql(
   3088             self,
   3089             name,

/usr/local/lib/python3.11/dist-packages/pandas/io/sql.py in
to_sql(frame, name, con, schema, if_exists, index, index_label,
chunksize, dtype, method, engine, **engine_kwargs)
    839         )
    840
--> 841     with pandasSQL_builder(con, schema=schema,
need_transaction=True) as pandas_sql:
    842         return pandas_sql.to_sql(
    843             frame,

/usr/local/lib/python3.11/dist-packages/pandas/io/sql.py in
pandasSQL_builder(con, schema, need_transaction)
    904
    905     if sqlalchemy is not None and isinstance(con, (str,
sqlalchemy.engine.Connectable)):
--> 906         return SQLDatabase(con, schema, need_transaction)
    907
    908     adbc =
import_optional_dependency("adbc_driver_manager.dbapi",
errors="ignore")

/usr/local/lib/python3.11/dist-packages/pandas/io/sql.py in
__init__(self, con, schema, need_transaction)
   1631         self.exit_stack = ExitStack()
   1632         if isinstance(con, str):
-> 1633             con = create_engine(con)
   1634             self.exit_stack.callback(con.dispose)
   1635         if isinstance(con, Engine):

/usr/local/lib/python3.11/dist-packages/sqlalchemy/engine/create.py in
create_engine(url, **kwargs)

/usr/local/lib/python3.11/dist-packages/sqlalchemy/util/deprecations.p
y in warned(fn, *args, **kwargs)
    279                         stacklevel=3,
    280                     )
```

```
--> 281             return fn(*args, **kwargs)  # type: ignore[no-any-
return]
    282
    283         doc = fn.__doc__ is not None and fn.__doc__ or ""

/usr/local/lib/python3.11/dist-packages/sqlalchemy/engine/create.py in
create_engine(url, **kwargs)
    547
    548     # create url.URL object
--> 549     u = _url.make_url(url)
    550
    551     u, plugins, kwargs = u._instantiate_plugins(kwargs)

/usr/local/lib/python3.11/dist-packages/sqlalchemy/engine/url.py in
make_url(name_or_url)
    854
    855     if isinstance(name_or_url, str):
--> 856         return _parse_url(name_or_url)
    857     elif not isinstance(name_or_url, URL) and not hasattr(
    858         name_or_url,
"_sqla_is_testing_if_this_is_a_mock_object"

/usr/local/lib/python3.11/dist-packages/sqlalchemy/engine/url.py in
_parse_url(name)
    920
    921     else:
--> 922         raise exc.ArgumentError(
    923             "Could not parse SQLAlchemy URL from string '%s'"
% name
    924         )

ArgumentError: Could not parse SQLAlchemy URL from string
'<connection>'
```

# Pandas Plotting

Pandas offers a convenient API for plotting data directly from a DataFrame. Of course, the plotting API is build upon `matplotlib` as a low level backend. We can use that to manipulate plots as in the previous lab. Check out the official documentation for visualization:

Pandas Plotting Docs

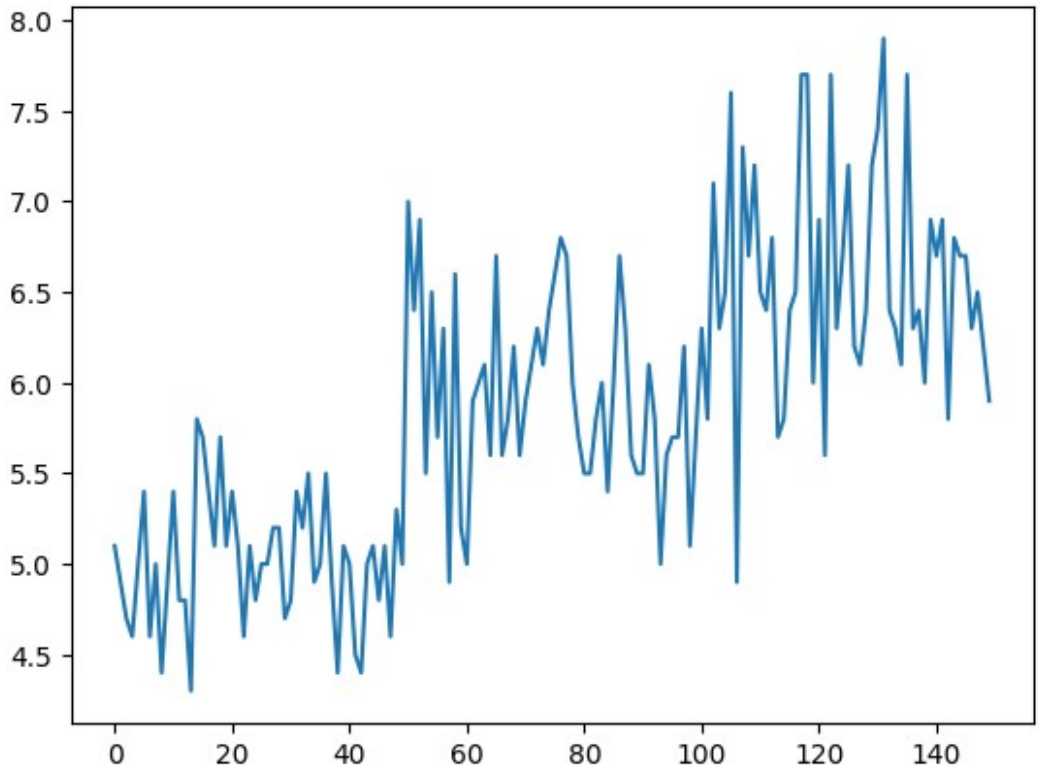
For a quick reference, check the official cookbook.

Pandas Plotting Cookbook

We will use the high level plotting API to visualize the Iris Dataset.

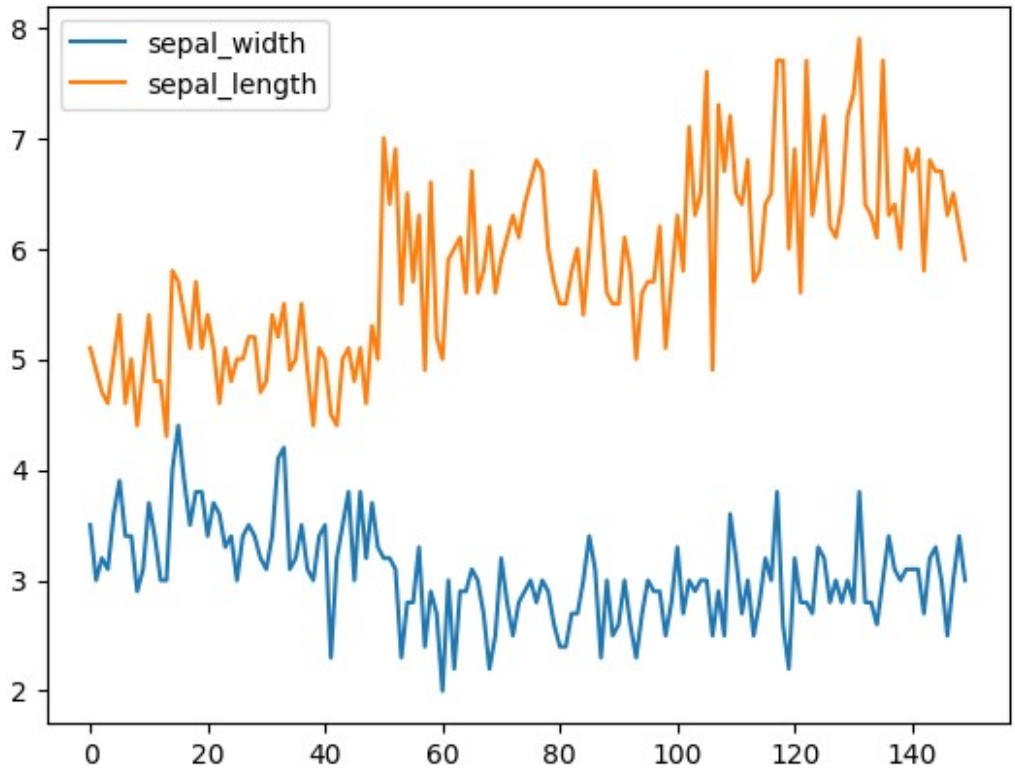
```
df['sepal_length'].plot()
```

```
<Axes: >
```




```
df[['sepal_width', 'sepal_length']].plot()
```

```
<Axes: >
```




```
fig, ax = plt.subplots(1, 2)

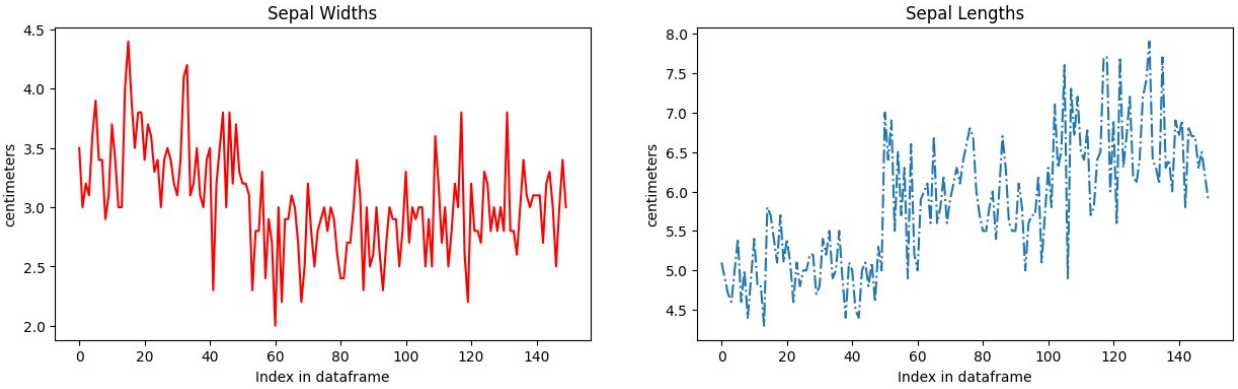
df['sepal_width'].plot(ax = ax[0], color = 'r')
df['sepal_length'].plot(ax = ax[1], linestyle = '-.')

ax[0].set_title('Sepal Widths')
ax[1].set_title('Sepal Lengths')

ax[0].set_xlabel('Index in dataframe')
ax[1].set_xlabel('Index in dataframe')

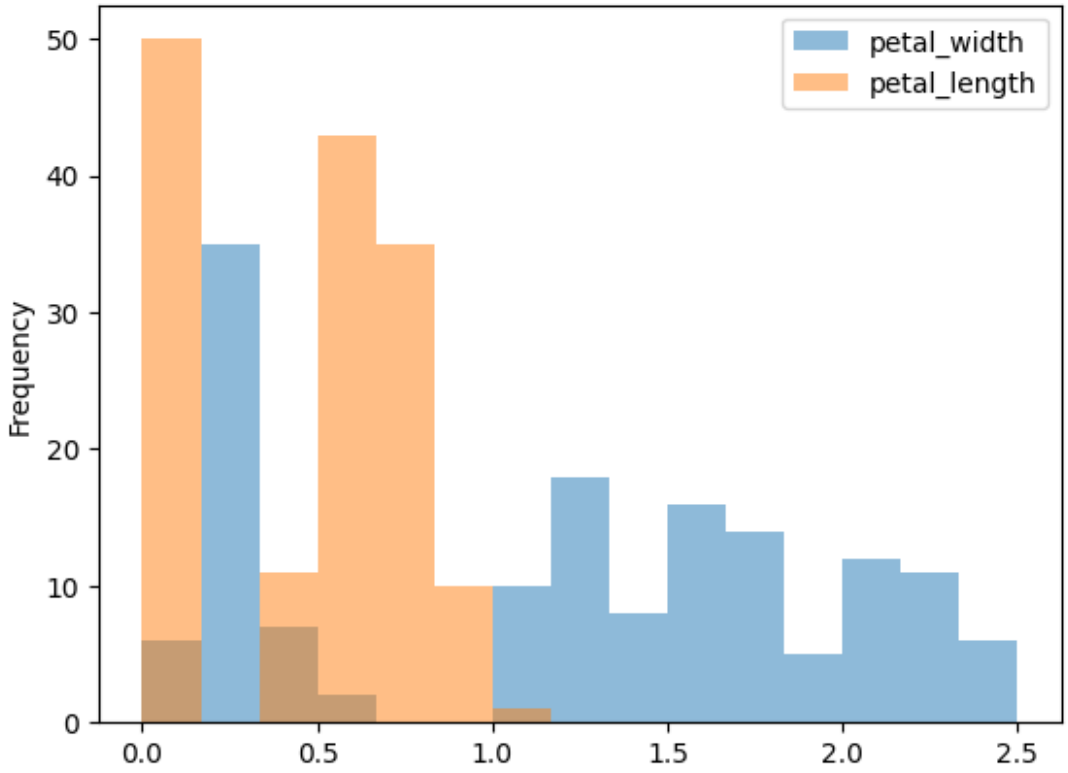
ax[0].set_ylabel('centimeters')
ax[1].set_ylabel('centimeters')


fig.set_size_inches(15, 4)
```



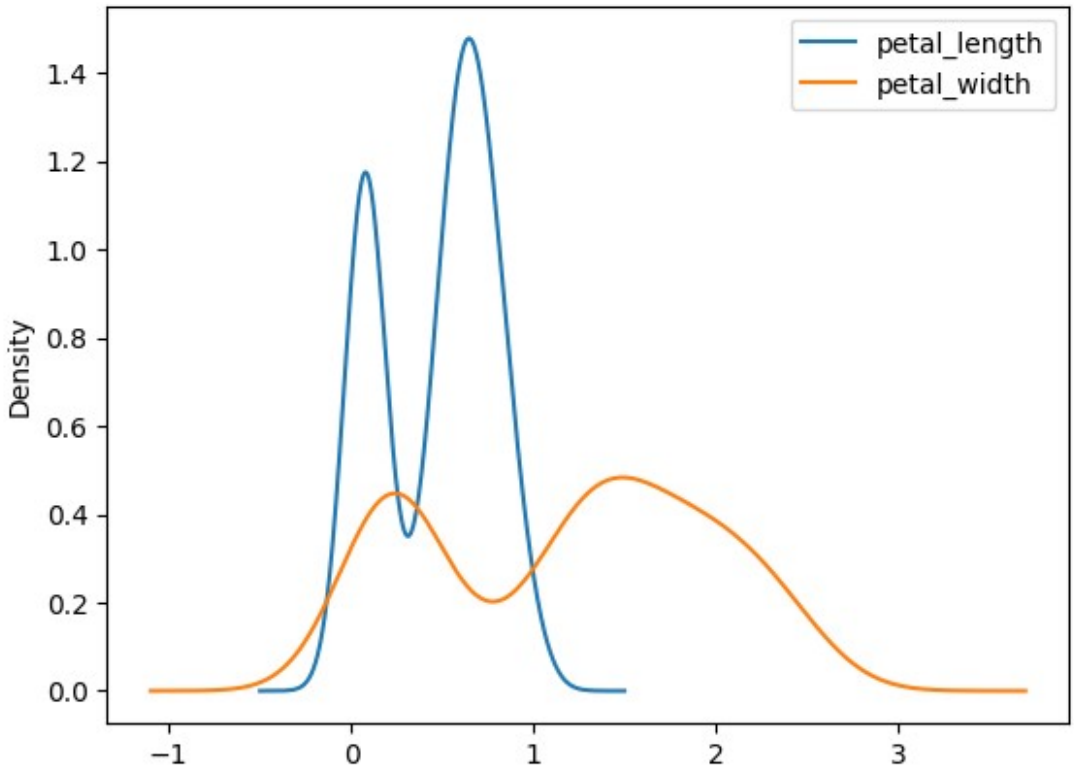

```
df[['petal_width', 'petal_length']].plot.hist(alpha = 0.5, bins = 15)
```

```
<Axes: ylabel='Frequency'>
```




```
df[['petal_length', 'petal_width']].plot.kde()
```

```
<Axes: ylabel='Density'>
```




## Cool Plots using Seaborn & Pandas

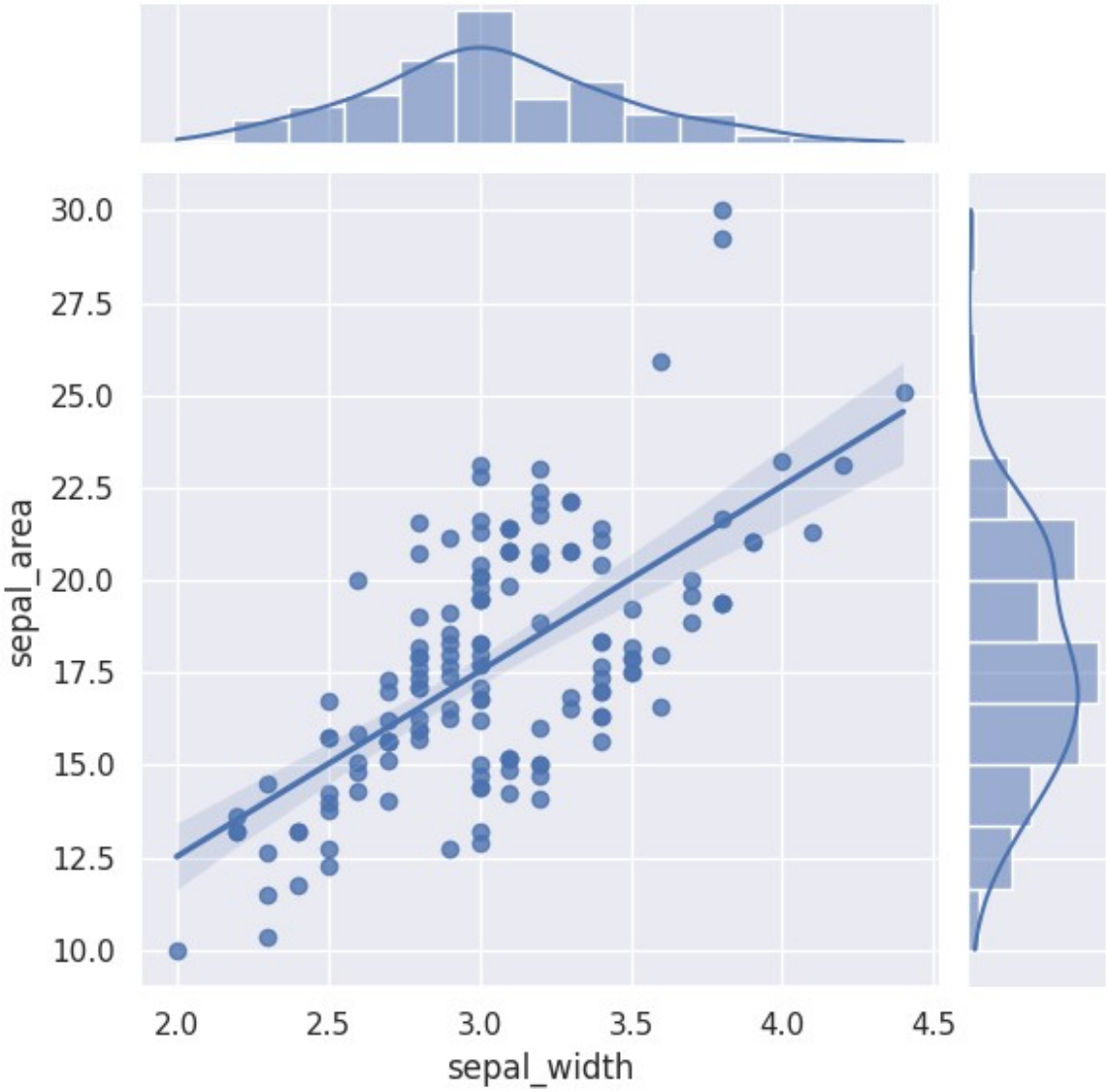
Check out seaborn for more awesome plots.

```
import seaborn as sns

sns.set_theme()

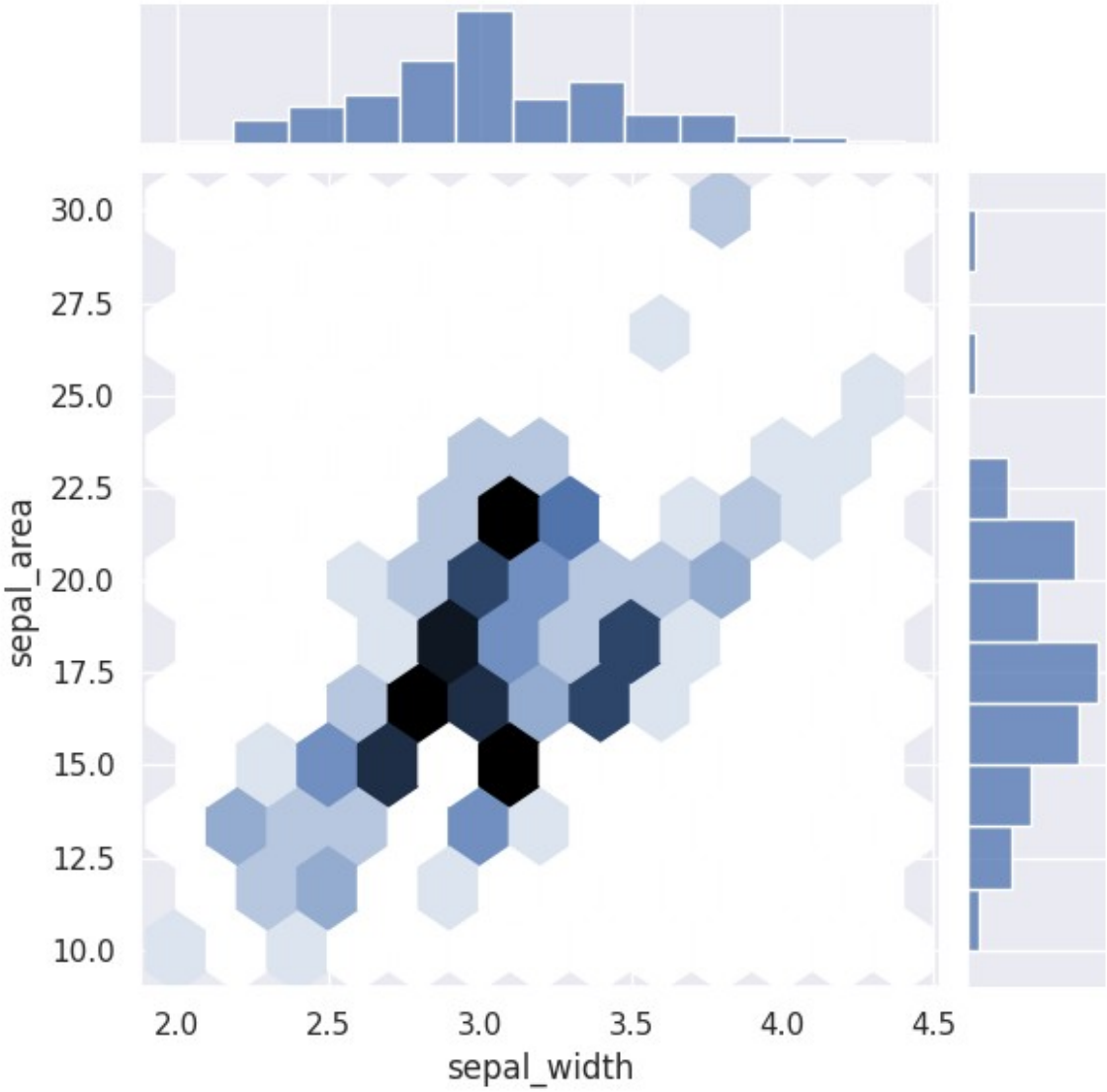
sns.jointplot(x = 'sepal_width', y = 'sepal_area', data = df, kind =
'reg')
```

```
<seaborn.axisgrid.JointGrid at 0x7f1cce30ce90>
```



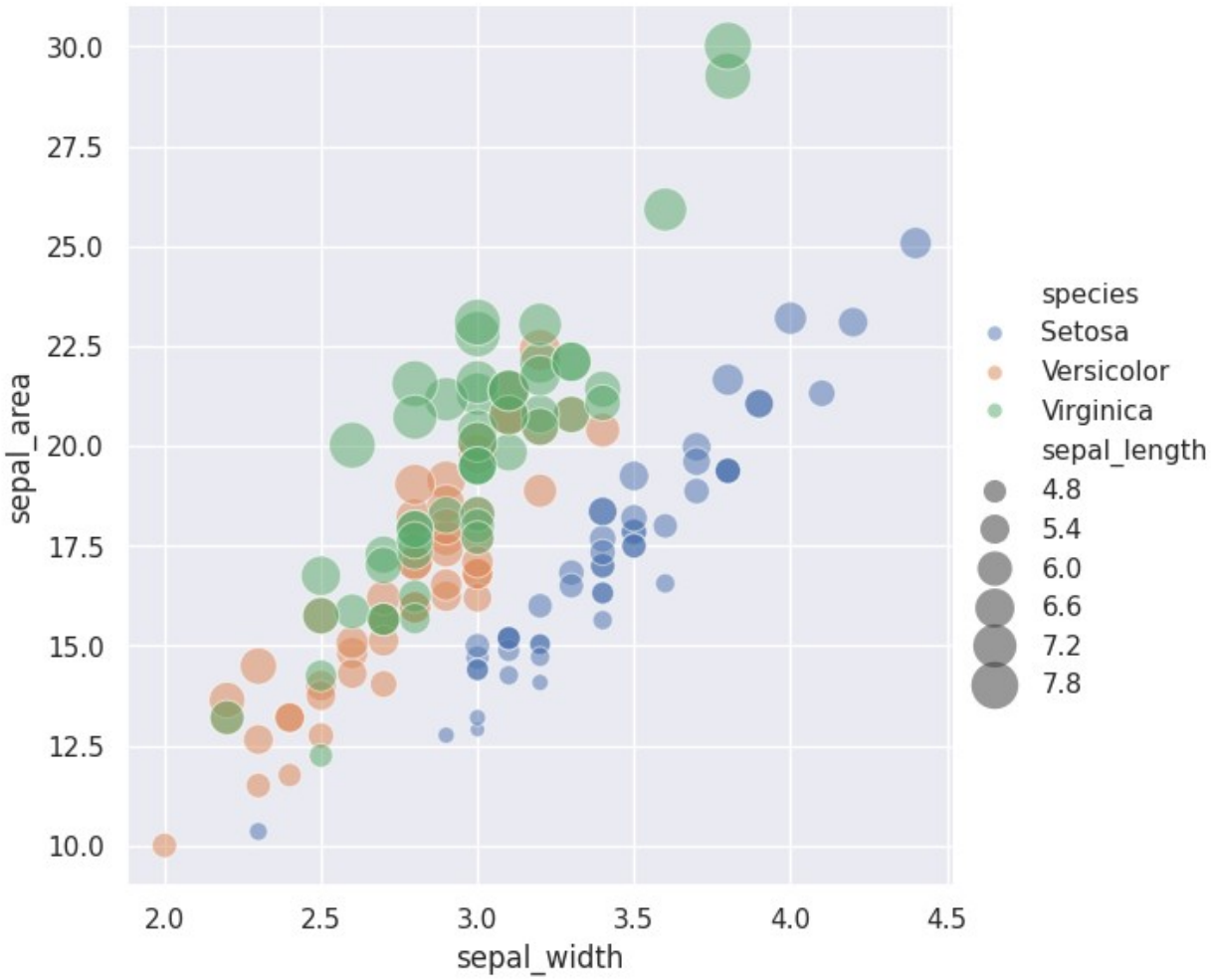

```
sns.jointplot(x = 'sepal_width', y = 'sepal_area', data = df, kind =
'hex')
```

```
<seaborn.axisgrid.JointGrid at 0x7f1cce0c9990>
```

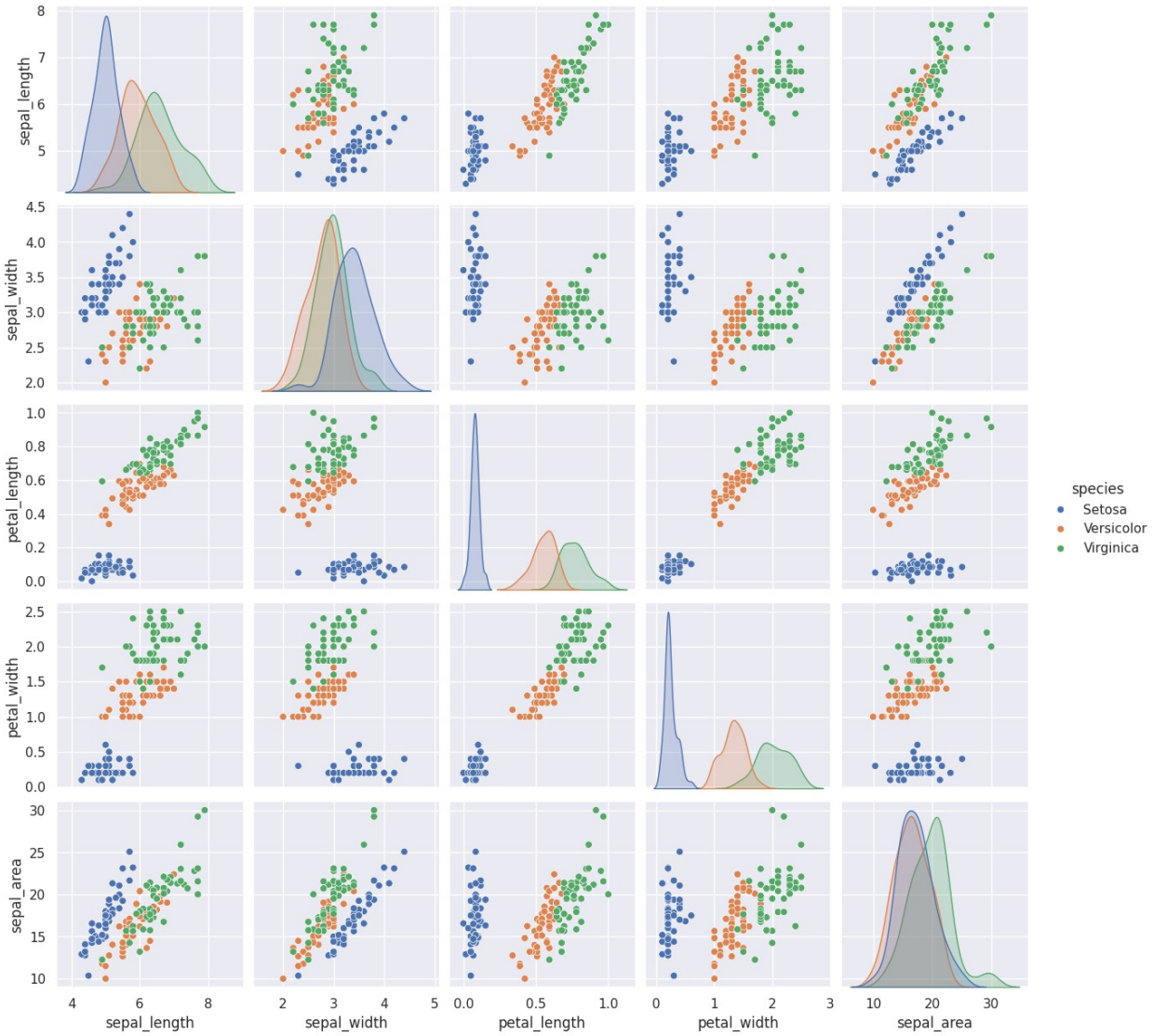



```
sns.relplot(x="sepal_width", y="sepal_area", hue="species",
size="sepal_length", sizes=(40, 400), alpha=.5, height=6, data=df)
```

```
<seaborn.axisgrid.FacetGrid at 0x7f1ccdf47790>
```

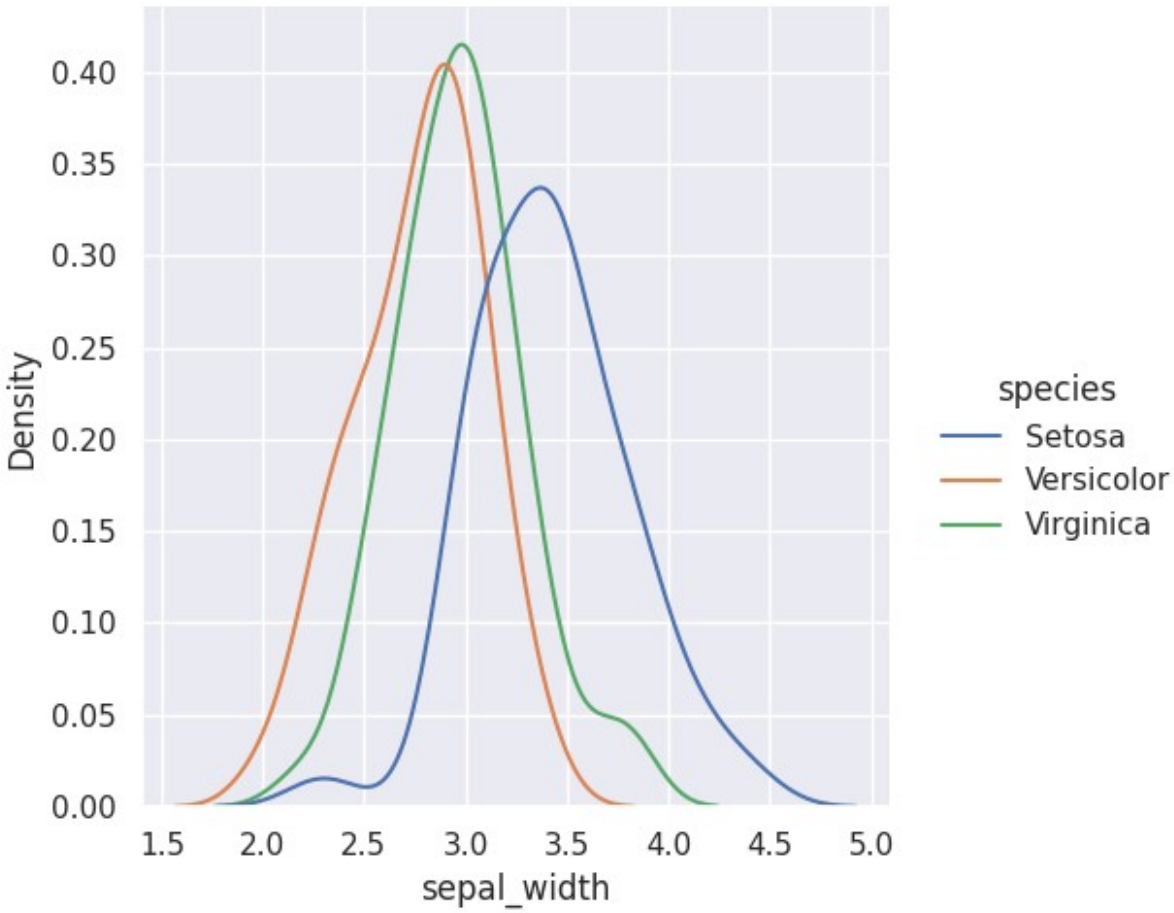



```
grid = sns.pairplot(df, hue = 'species')
```



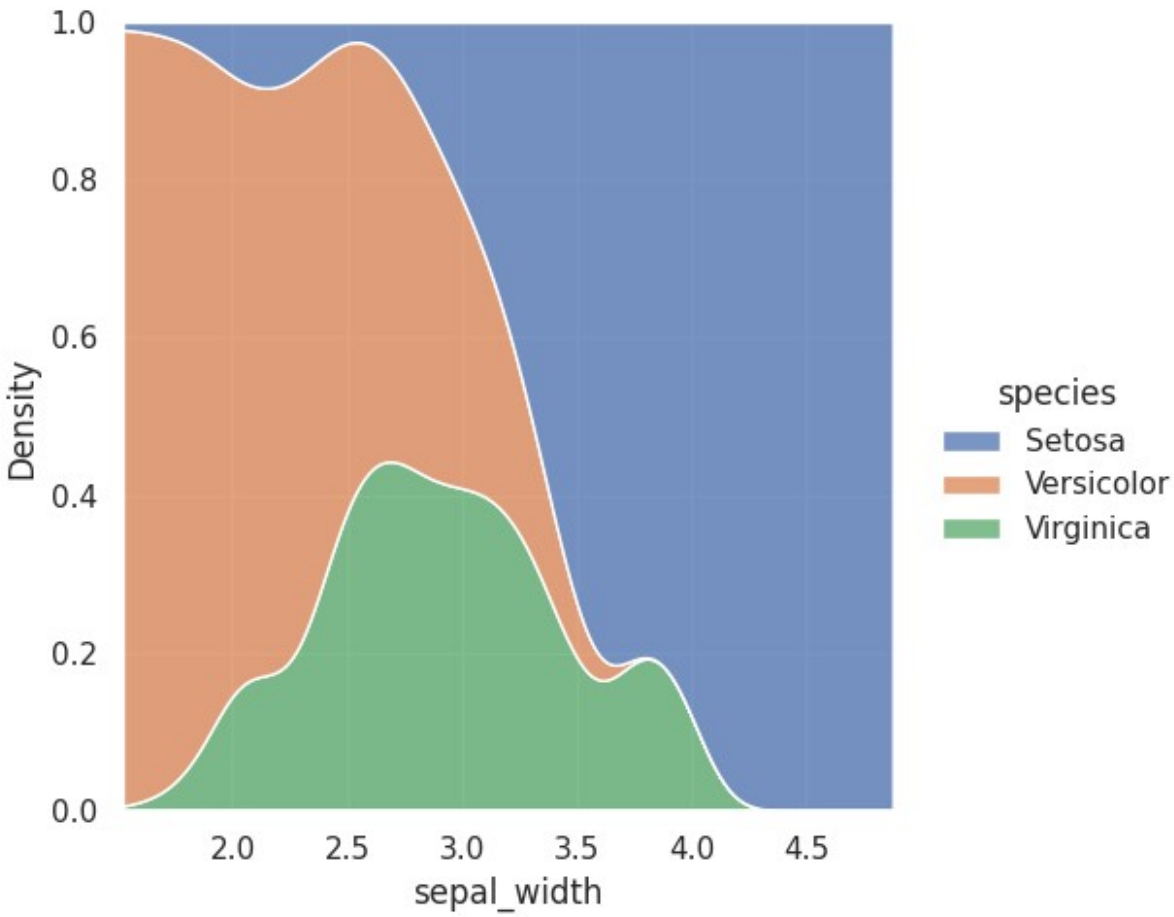

```
sns.displot(data = df, x = 'sepal_width', kind = 'kde', hue =
'species')
```

```
<seaborn.axisgrid.FacetGrid at 0x7f1cccf97c90>
```

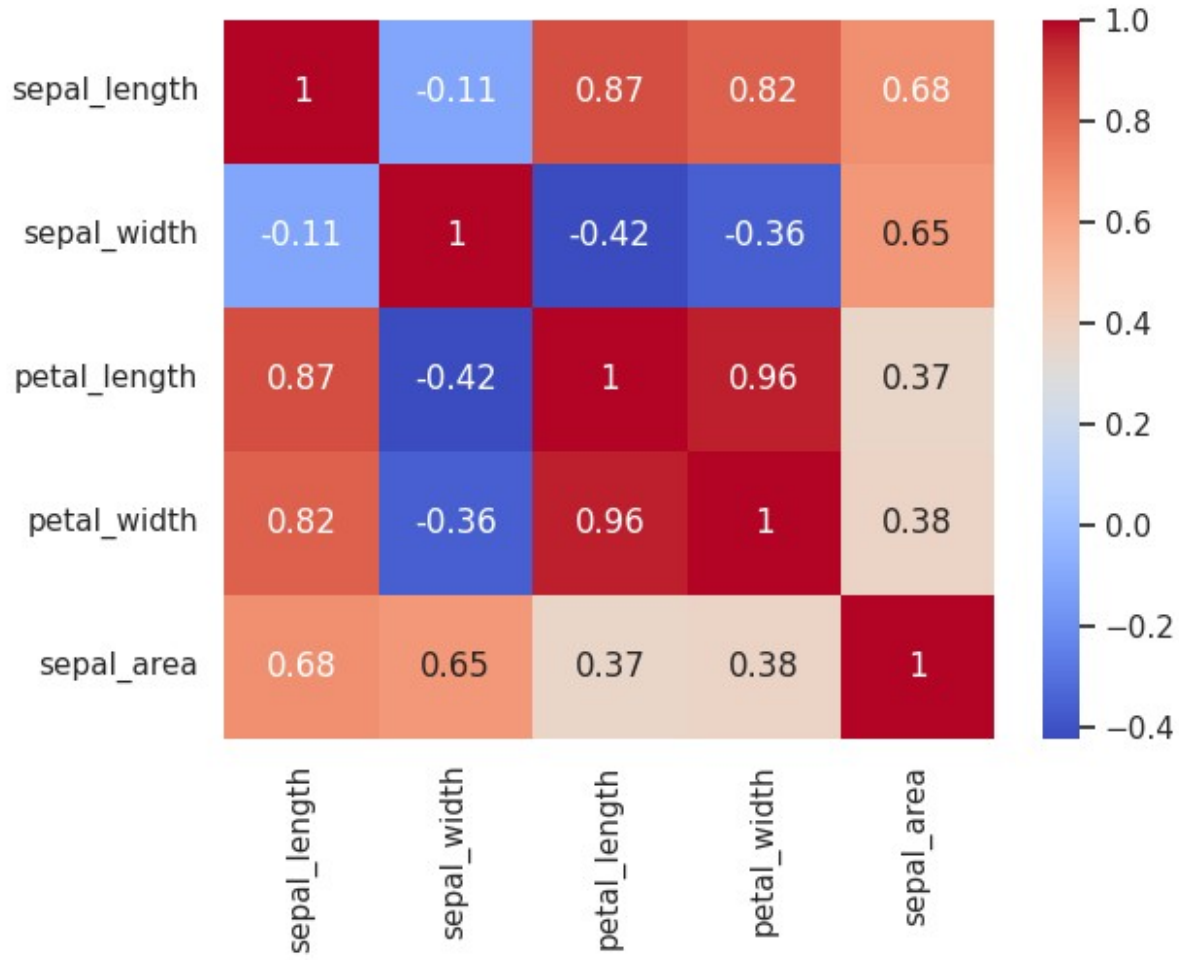



```
sns.displot(data = df, x = 'sepal_width', kind = 'kde', hue =
'species', multiple = 'fill')
```

```
<seaborn.axisgrid.FacetGrid at 0x7f1cccc3af10>
```

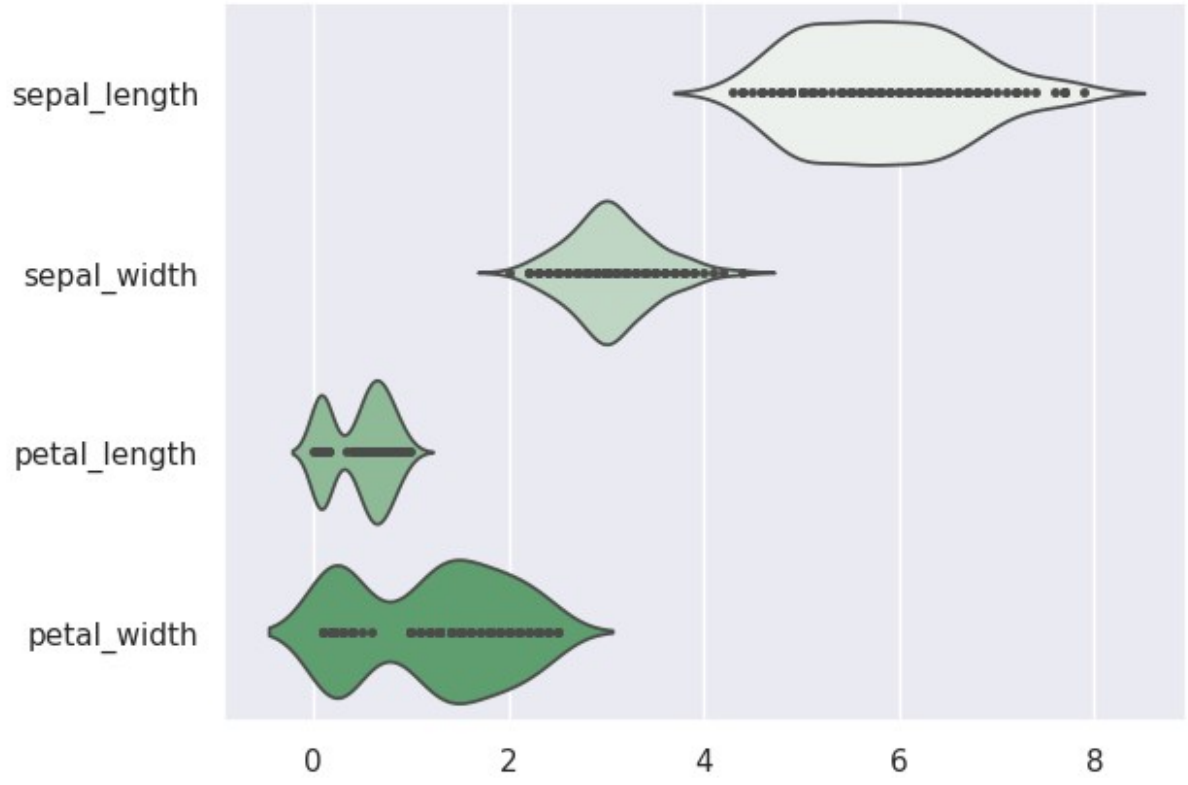



```
sns.heatmap(df.select_dtypes(include=['number']).corr(), annot=True,
cmap="coolwarm")
plt.show()
```




```
sns.violinplot(data=df[['sepal_length', 'sepal_width', 'petal_length',
'petal_width']], palette="light:g", inner="points", orient="h")
```

```
<Axes: >
```



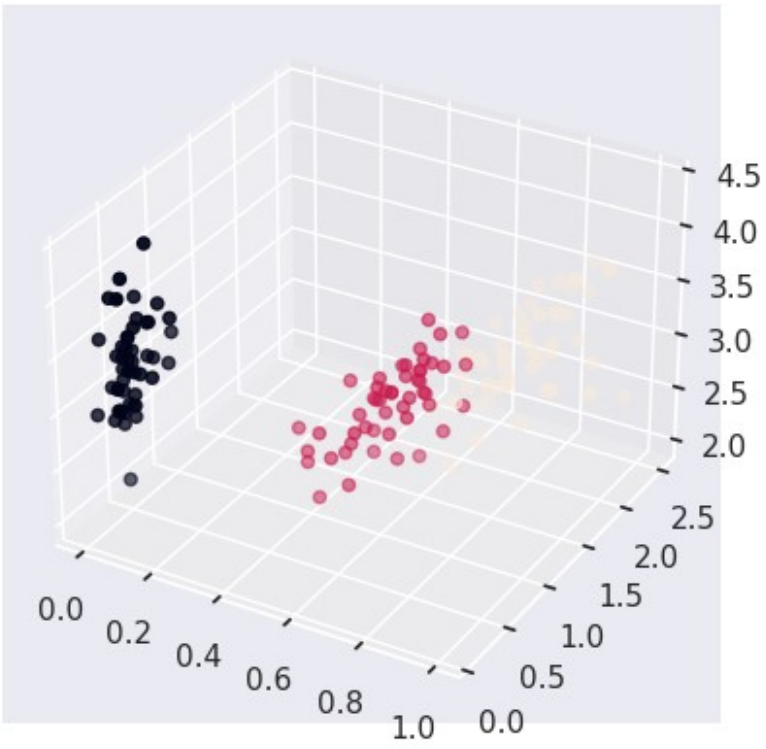

## 3D Plotting

```
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.scatter(df['petal_length'], df['petal_width'], df['sepal_width'],
zdir='z', s=20, c=df['species'].astype('category').cat.codes,
depthshade=True)
```

```
<mpl_toolkits.mplot3d.art3d.Path3DCollection at 0x7f1ccccdd3d0>
```




# Exercises

After you finish your lab exercises, you should export this notebook as **pdf** and upload it to Moodle. (i.e. **File -> Print**, Destintation: Save as PDF).

In this lab, we will look at COVID data for Romania. The data was retrieved from https://graphs.ro/.

```
import requests
import json

covid_data =
json.loads(requests.get('https://www.graphs.ro/json.php').content)

covid_df = pd.DataFrame(covid_data['covid_romania'])
covid_df.head()

covid_county_data_dfs = []

for i, row in covid_df.iterrows():
  try: # some days have no county information
    county_df = pd.DataFrame(row['county_data'])
  except:
    continue
```

```
  county_df['reporting_date'] = row['reporting_date']
  covid_county_data_dfs.append(county_df)


county_df = pd.concat(covid_county_data_dfs)
covid_df = covid_df.drop(['sourceUrl', 'county_data'], axis = 1)

covid_df.head()
```


```
{"type":"dataframe","variable_name":"covid_df"}
```


```
county_df.head()
```


```
{"summary":"{\n  \"name\": \"county_df\",\n  \"rows\": 40488,\n
\"fields\": [\n    {\n      \"column\": \"county_id\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 42,\n        \"samples\": [\n          \"MM\",\
n          \"CT\",\n          \"BR\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"county_name\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 42,\n        \"samples\": [\n
\"Maramures\",\n          \"Constanta\",\n          \"Braila\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"county_population\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
265641,\n        \"min\": 192037,\n        \"max\": 1832883,\n
\"num_unique_values\": 42,\n        \"samples\": [\n          458471,\
n          671910,\n          285851\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"total_cases\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
58876,\n        \"min\": 0,\n        \"max\": 633881,\n
\"num_unique_values\": 26810,\n        \"samples\": [\n
9951,\n          16062,\n          12808\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"reporting_date\",\n
\"properties\": {\n        \"dtype\": \"date\",\n        \"min\":
\"2020-04-02\",\n        \"max\": \"2022-11-21\",\n
\"num_unique_values\": 964,\n        \"samples\": [\n          \"2020-
10-22\",\n          \"2020-06-20\",\n          \"2020-04-27\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    }\n  ]\n}","type":"dataframe","variable_name":"county_df"}
```


# 1. Basic Visualizations

Make 4 subplots. Using pandas as seaborn, plot the number of new cases in a day, the number of recovered patients in day, number of tests in a day, and the number of deaths in day. We are trying to explore the evolution of COVID from the start of the pandemic until today.

**NB:** Make sure to add proper labels, title, axes and legend where necessary.

```
# TODO your code here
# incarc datele (am incercat direct, n-a mers, trebuie sa fac asta de
fiecare data)
covid_data =
json.loads(requests.get('https://www.graphs.ro/json.php').content)
covid_df = pd.DataFrame(covid_data['covid_romania'])

# trebuie sa convertesc la date time ca sa mearga asta cu zile
covid_df['reporting_date'] =
pd.to_datetime(covid_df['reporting_date'])

# pentru ca e pe zile, tre sa indexez dupa data
covid_df.set_index('reporting_date', inplace=True)

# covid_df.head() # asta ca sa stiu ce sa selectez

# Creez 4 subploturi
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
# Cazuri noi
sns.lineplot(ax=axes[0, 0], x=covid_df.index,
y=covid_df['new_cases_today'])
# selectez chestii cu today ca fac pe zile si iau pe zile
axes[0, 0].set_title("Cazuri / zi")
axes[0, 0].set_xlabel("Luna-an")
axes[0, 0].set_ylabel("Nr. indivizi")

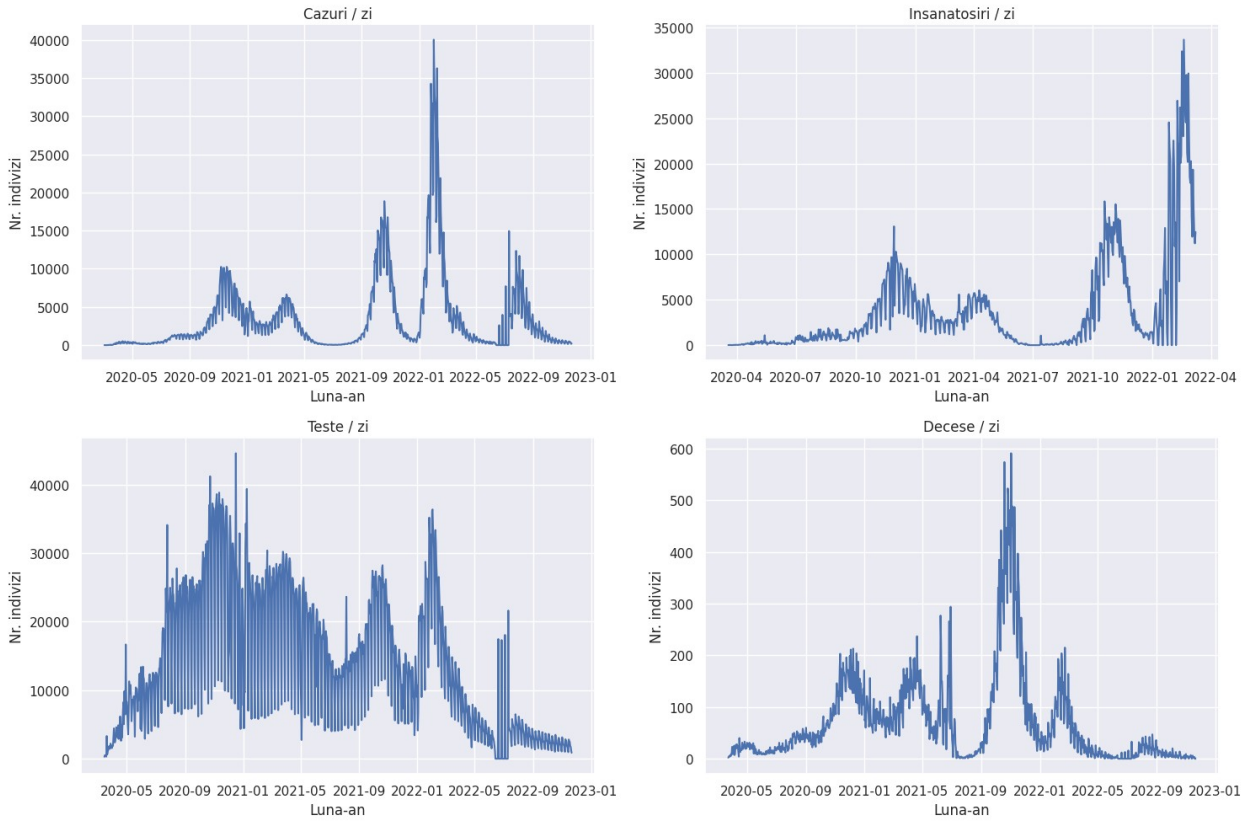
# Insanatosiri
sns.lineplot(ax=axes[0, 1], x=covid_df.index,
y=covid_df['new_recovered_today'])
axes[0, 1].set_title("Insanatosiri / zi")
axes[0, 1].set_xlabel("Luna-an")
axes[0, 1].set_ylabel("Nr. indivizi")

# Numar de teste
sns.lineplot(ax=axes[1, 0], x=covid_df.index,
y=covid_df['new_tests_today'])
axes[1, 0].set_title("Teste / zi")
axes[1, 0].set_xlabel("Luna-an")
axes[1, 0].set_ylabel("Nr. indivizi")

# Decese
sns.lineplot(ax=axes[1, 1], x=covid_df.index,
y=covid_df['new_deaths_today'])
axes[1, 1].set_title("Decese / zi")
axes[1, 1].set_xlabel("Luna-an")
axes[1, 1].set_ylabel("Nr. indivizi")

# Afisare grafice
```

```
plt.tight_layout()
plt.show()
```




# 2. Positive testing percentage

Using `pandas`, create a new column that computes the percentage of positive tests in a given day. This new column should be the number of infected people in a day over the number of tests per day.

Plot the evolution of positive tests across time. Compare this to the number of hospitalized patients.

In a different plot, visualize the correlation between positive tests and the number of intensive care patients.

**NB:** Make sure to add proper labels, title, axes and legend where necessary.

```
# TODO your code here
# ibidem inceputul, luat din celula de mai sus
# incarc datele (am incercat direct, n-a mers, trebuie sa fac asta de
fiecare data)
covid_data =
json.loads(requests.get('https://www.graphs.ro/json.php').content)
```

```
covid_df = pd.DataFrame(covid_data['covid_romania'])

# trebuie sa convertesc la date time ca sa mearga asta cu zile
covid_df['reporting_date'] =
pd.to_datetime(covid_df['reporting_date'])

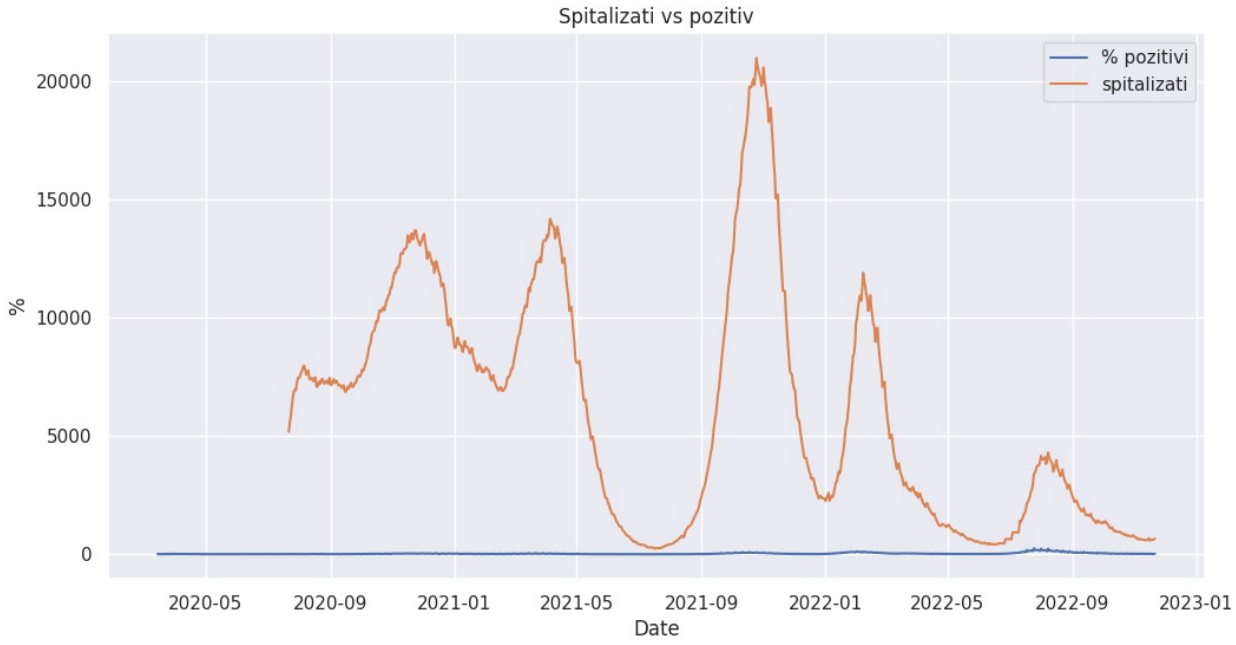
# pentru ca e pe zile, tre sa indexez dupa data
covid_df.set_index('reporting_date', inplace=True)

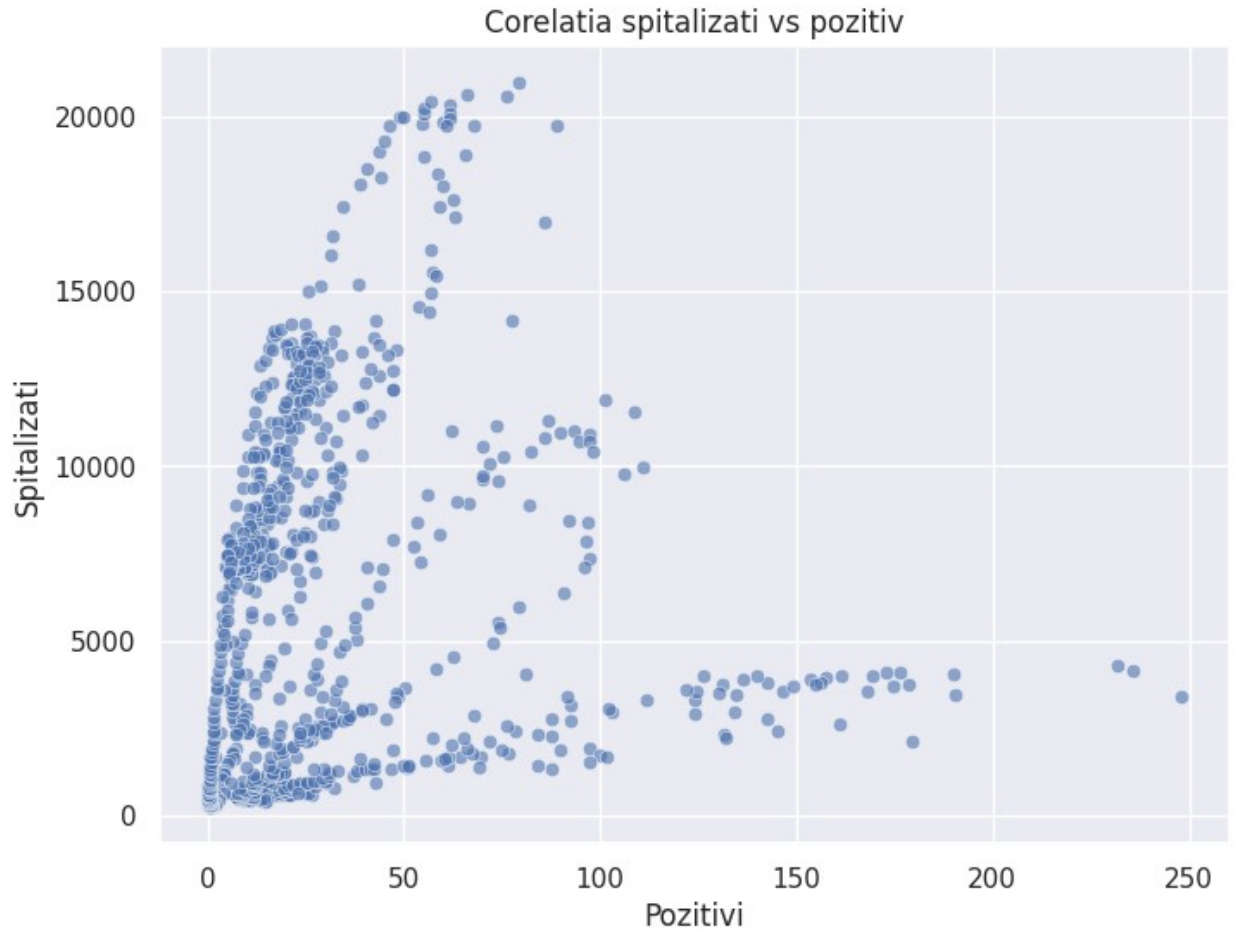
# procentul calculat conform indicatiei de mai sus
covid_df['procent_1'] = (covid_df['new_cases_today'] /
covid_df['new_tests_today']) * 100

# ma intreb cand voi scapa de asta, daca voi invata capul de tbel =))
covid_df.head()

# Acum afisez
plt.figure(figsize=(12, 6))
sns.lineplot(data=covid_df, x=covid_df.index, y='procent_1', label='%
pozitivi')
sns.lineplot(data=covid_df, x=covid_df.index,
y='infected_hospitalized', label='spitalizati') # ???
plt.title("Spitalizati vs pozitiv")
plt.xlabel("Date")
plt.ylabel("%")
plt.legend()
plt.show()

# corelatie
plt.figure(figsize=(8, 6))
sns.scatterplot(data=covid_df, x='procent_1',
y='infected_hospitalized', alpha=0.6)
plt.title("Corelatia spitalizati vs pozitiv")
plt.xlabel("Pozitivi")
plt.ylabel("Spitalizati")
plt.show()
```


Spitalizati vs pozitiv
% pozitivi
spitalizati
20000
15000
10000
5000
0
%
2020-05
2020-09
2021-01
2021-05
2021-09
2022-01
2022-05
2022-09
2023-01
Date


Corelatia spitalizati vs pozitiv
20000
15000
10000
5000
0
Spitalizati
0
50
100
150
200
250
Pozitivi

# 3. County Information

Select at least 10 counties of your choosing, and plot the evolution of cases across time.

Plot the number of positive cases on 1000 persons for each of your selected counties. Plot a horizontal line at the 3 / 1000 mark. When the number of positive cases per 1000 persons exceeds 3 / 1000, color your points in a different color from that point onwards.

**NB:** Make sure to add proper labels, title, axes and legend where necessary.

```
# ibidem inceputul, luat din celula de mai sus
# incarc datele (am incercat direct, n-a mers, trebuie sa fac asta de
fiecare data)
covid_data =
json.loads(requests.get('https://www.graphs.ro/json.php').content)
covid_df = pd.DataFrame(covid_data['covid_romania'])

# trebuie sa convertesc la date time ca sa mearga asta cu zile
covid_df['reporting_date'] =
pd.to_datetime(covid_df['reporting_date'])

# pentru ca e pe zile, tre sa indexez dupa data
covid_df.set_index('reporting_date', inplace=True)

# procentul calculat conform indicatiei de mai sus
covid_df['procent_1'] = (covid_df['new_cases_today'] /
covid_df['new_tests_today']) * 100

# incerc sa fac rost de date pe judet
covid_county_data_dfs = []

for i, row in covid_df.iterrows():
  try: # e posibil sa nu existe date pentru anumite judete
    county_df = pd.DataFrame(row['county_data'])
  except:
    continue

  county_df['reporting_date'] = i
  covid_county_data_dfs.append(county_df)

county_df = pd.concat(covid_county_data_dfs)
covid_df = covid_df.drop(['sourceUrl', 'county_data'], axis=1)

print("Coloanele disponibile în county_df:")
print(county_df.columns)

# incerc sa aflu top 10 județe după numărul total de cazuri la data
cea mai recentă
latest_date = county_df['reporting_date'].max()
latest_data = county_df[county_df['reporting_date'] == latest_date]
```

```
top_counties = latest_data.sort_values('total_cases',
ascending=False).head(10)

# extrag numele celor 10 județe cu cele mai multe cazuri
selected_counties = top_counties['county_name'].tolist()
print("\nTop 10 județe selectate automat după numărul de cazuri:")
print(selected_counties)

# iau datele doar pentru aceste județe
county_df_selected =
county_df[county_df['county_name'].isin(selected_counties)]

# calculez rata de cazuri per 1000 de locuitori folosind direct
county_population din dataset, pentru a face niste rapoarte
county_df_selected['cases_per_1000'] =
(county_df_selected['total_cases'] /
county_df_selected['county_population']) * 1000

# afisez la fiecare 1000 de locuitori, gen la mie
plt.figure(figsize=(14, 8))
for county in selected_counties:
    subset = county_df_selected[county_df_selected['county_name'] ==
county]
    plt.scatter(subset['reporting_date'], subset['cases_per_1000'],
label=county, alpha=0.6, s=30)
    # adaug o linie pentru a urmări mai ușor evoluția
    plt.plot(subset['reporting_date'], subset['cases_per_1000'],
alpha=0.4)

plt.axhline(y=3, color='red', linestyle='--', label='3 cases/1000
threshold')

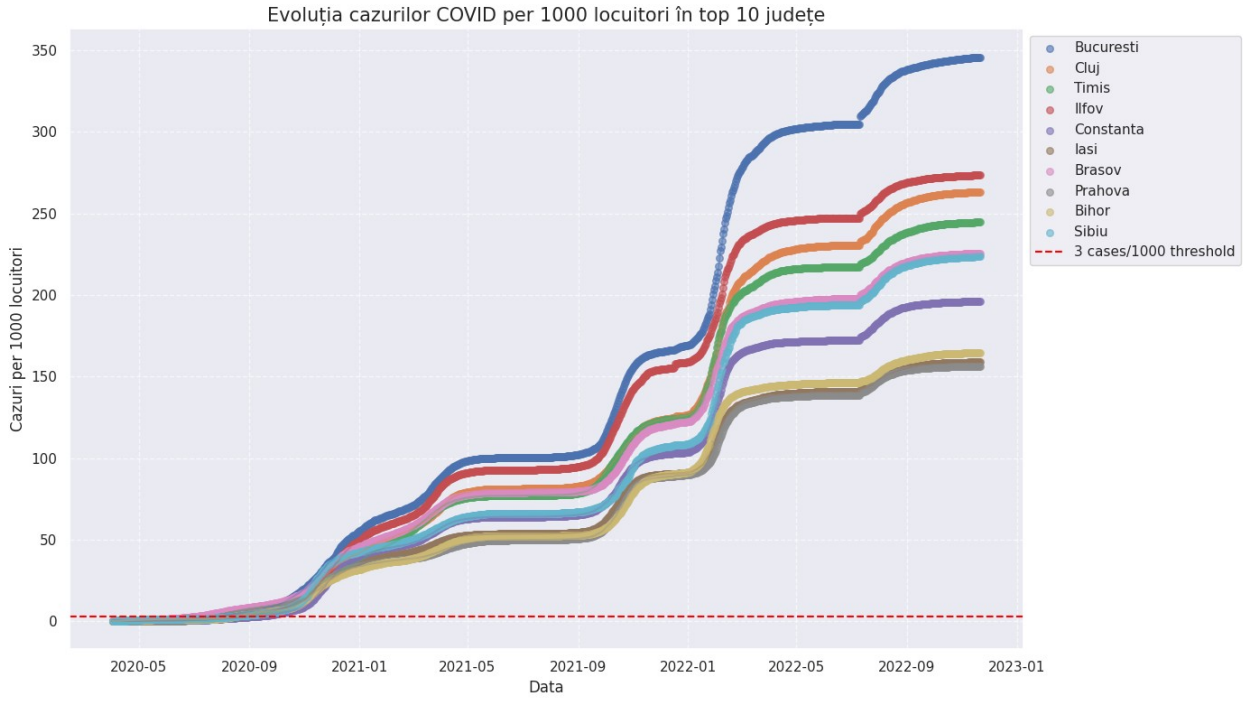
plt.title("Evoluția cazurilor COVID per 1000 locuitori în top 10
județe", fontsize=15)
plt.xlabel("Data", fontsize=12)
plt.ylabel("Cazuri per 1000 locuitori", fontsize=12)
plt.legend(loc='upper left', bbox_to_anchor=(1, 1))
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

Coloanele disponibile în county_df:
Index(['county_id', 'county_name', 'county_population', 'total_cases',
       'reporting_date'],
      dtype='object')

Top 10 județe selectate automat după numărul de cazuri:
['Bucuresti', 'Cluj', 'Timis', 'Ilfov', 'Constanta', 'Iasi', 'Brasov',
'Prahova', 'Bihor', 'Sibiu']
```

```
<ipython-input-7-06a318ff4c63>:47: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  county_df_selected['cases_per_1000'] =
(county_df_selected['total_cases'] /
county_df_selected['county_population']) * 1000
```




# BONUS

Further expore the dataset, and come up with interesting visualizations of the COVID evolution in Romania.

```
# TODO your code here (maybe)

# 1. Analiză de sezonalitate - Compar tiparele de infectare pe
anotimpuri
monthly_cases = covid_df.resample('M')['new_cases_today'].sum()
plt.figure(figsize=(14, 6))
sns.lineplot(x=monthly_cases.index, y=monthly_cases.values,
marker='o')
plt.title('Evoluția lunară a cazurilor COVID în România', fontsize=15)
plt.xlabel('Data')
```

```
plt.ylabel('Cazuri noi lunare')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# 2. Heatmap al incidenței COVID pe județe și timp
pivot_data = county_df.pivot_table(
    index='reporting_date',
    columns='county_name',
    values='total_cases',
    aggfunc='max'
).fillna(method='ffill')

# Calculez rata de creștere săptămânală
growth_rate = pivot_data.pct_change(periods=7) * 100

# iau ultimele 12 luni din date și top 15 județe (o varianta mai
simpla de a lua topul =)) )
recent_months = growth_rate.tail(52)
top_counties = pivot_data.iloc[-
1].sort_values(ascending=False).head(15).index

# Heatmap cu valorile absolute ale cazurilor COVID
plt.figure(figsize=(16, 10))
sns.heatmap(
    pivot_data.tail(52)[top_counties].T,
    cmap='YlOrRd',
    cbar_kws={'label': 'Număr total de cazuri'}
)
plt.title('Evoluția cazurilor COVID absolute pe județe', fontsize=16)
plt.tight_layout()
plt.show()

# 3. Comparație între rata de testare și rata de pozitivare (cu media
normală)
plt.figure(figsize=(14, 8))
ax1 = plt.gca()
ax2 = ax1.twinx()

# Calculez media normală pentru întreaga perioadă
mean_tests = covid_df['new_tests_today'].mean()
mean_cases = covid_df['new_cases_today'].mean()
mean_positivity = (mean_cases / mean_tests * 100)

# afisez valorile zilnice
ax1.plot(covid_df.index, covid_df['new_tests_today'], 'b-', alpha=0.4,
label='Teste zilnice')
ax1.axhline(y=mean_tests, color='b', linestyle='-', linewidth=2,
label=f'Medie teste: {mean_tests:.0f}')
ax1.set_ylabel('Număr de teste', color='b')
```

```
ax1.tick_params(axis='y', colors='b')

# calculez rata de pozitivare zilnică
daily_positivity = (covid_df['new_cases_today'] /
covid_df['new_tests_today'] * 100)
ax2.plot(covid_df.index, daily_positivity, 'r-', alpha=0.4,
label='Rată de pozitivare zilnică (%)')
ax2.axhline(y=mean_positivity, color='r', linestyle='-', linewidth=2,
label=f'Medie pozitivare: {mean_positivity:.2f}%')
ax2.set_ylabel('Rată de pozitivare (%)', color='r')
ax2.tick_params(axis='y', colors='r')
ax2.axhline(y=5, color='r', linestyle='--', alpha=0.5, label='Prag OMS
(5%)')

# combin legendele
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper left')

plt.title('Relația dintre testare și rata de pozitivare în timp (cu
medie normală)', fontsize=15)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# 4. Analiza geografică a impactului (bubble chart corectat)
plt.figure(figsize=(14, 10))
latest_data = county_df[county_df['reporting_date'] ==
county_df['reporting_date'].max()]

# Calculăm cases_per_1000 direct în cod (nu folosim o coloană
existentă)
cases_per_1000 = latest_data['total_cases'] /
latest_data['county_population'] * 1000

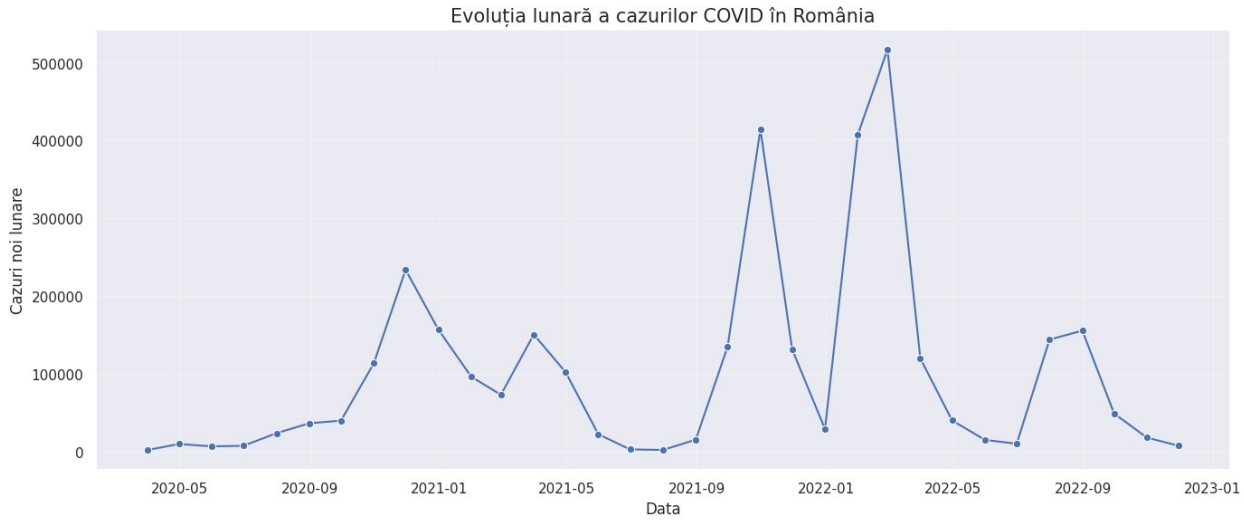
plt.scatter(
    latest_data['county_population'],
    latest_data['total_cases'],
    s=cases_per_1000 * 30,  # Mărimea bubble calculată direct
    alpha=0.6,
    c=cases_per_1000,  # Culoare bazată pe cases_per_1000 calculată
direct
    cmap='YlOrRd'
)

# Adaug etichete pentru fiecare județ
for i, row in latest_data.iterrows():
    cases_per_1000_value = row['total_cases'] /
row['county_population'] * 1000
    plt.annotate(
```

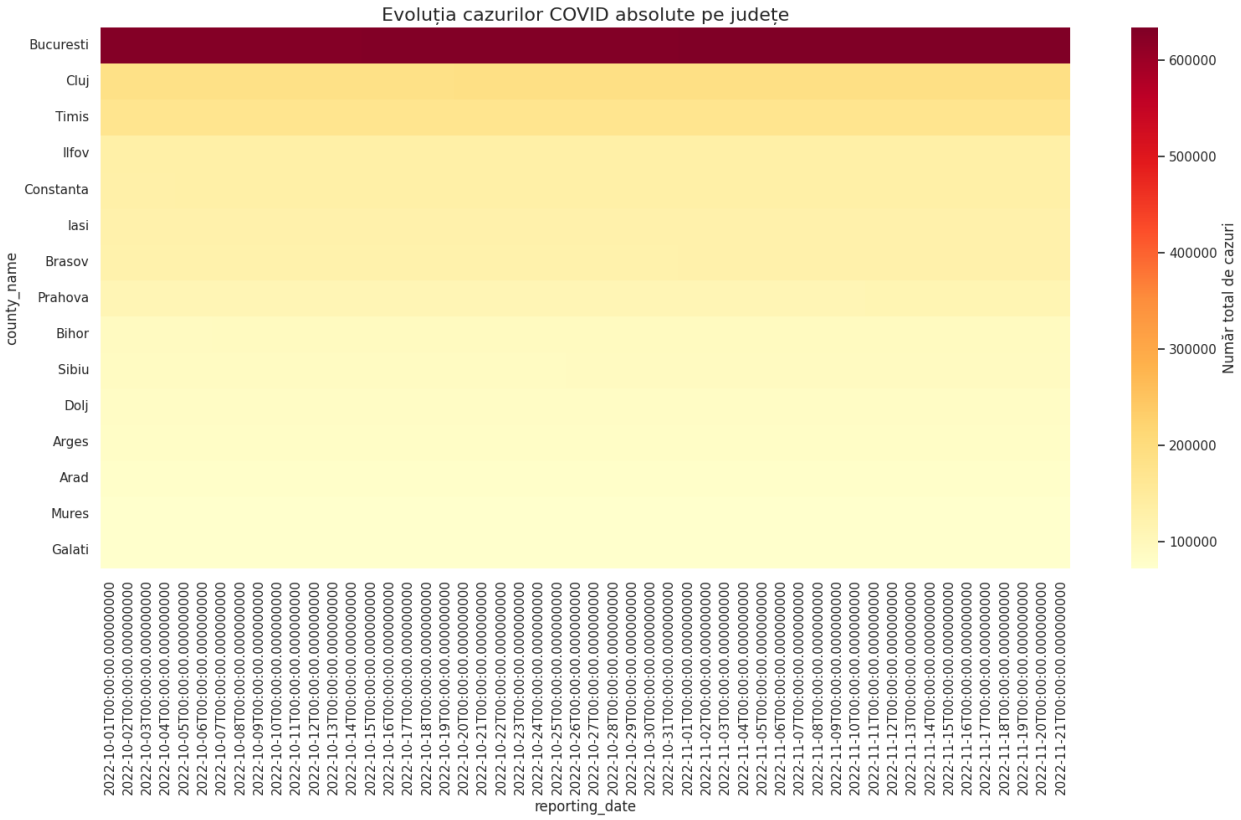
```
        f"{row['county_name']} ({cases_per_1000_value:.1f})",
        (row['county_population'], row['total_cases']),
        xytext=(5, 5),
        textcoords='offset points',
        fontsize=8
    )

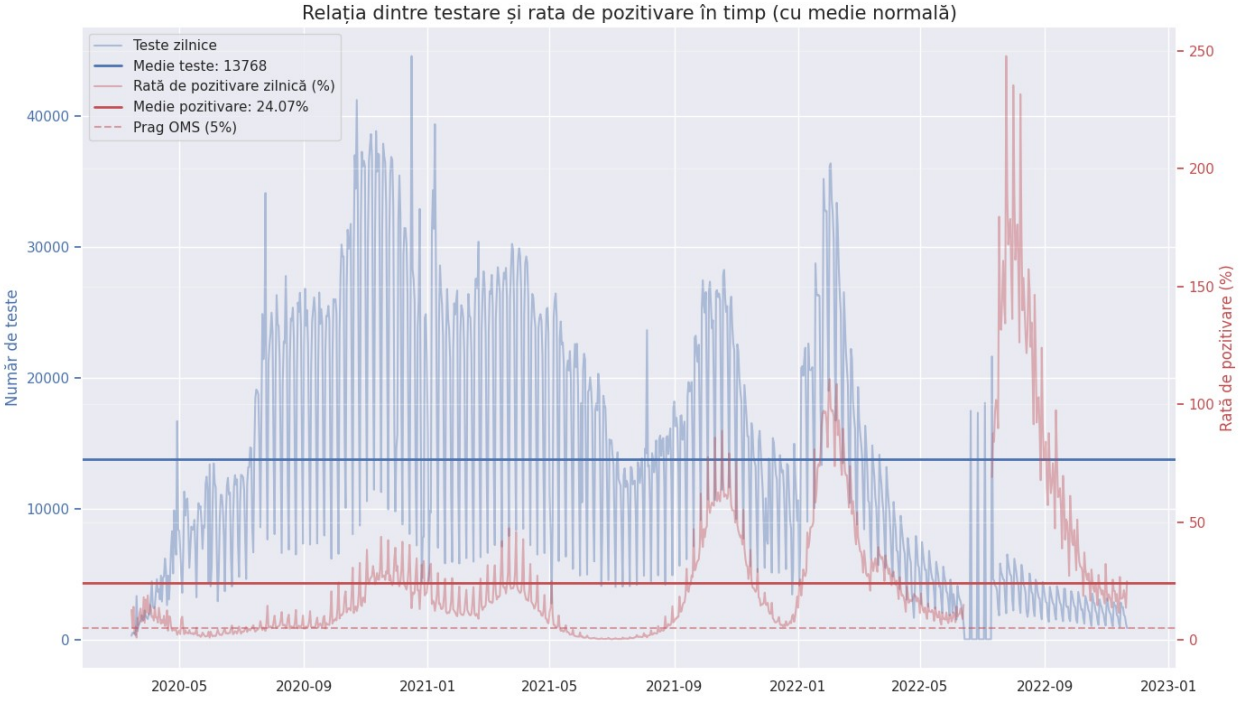
plt.colorbar(label='Cazuri la 1000 locuitori')
plt.xlabel('Populație județ')
plt.ylabel('Total cazuri COVID')
plt.title('Distribuția geografică a cazurilor COVID în raport cu
populația', fontsize=15)
plt.xscale('log')
plt.yscale('log')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

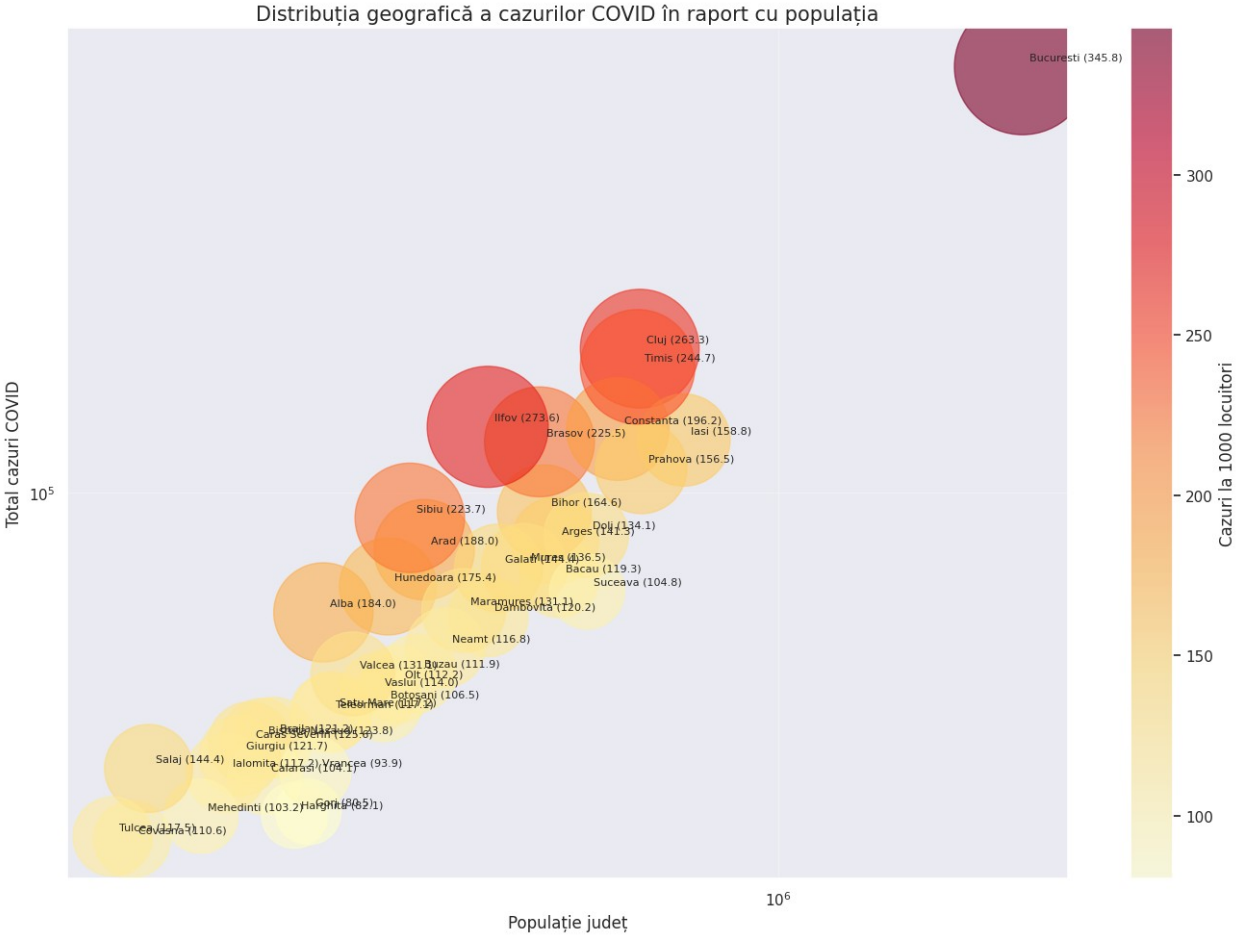
```
<ipython-input-16-b4f08fe1d526>:4: FutureWarning: 'M' is deprecated
and will be removed in a future version, please use 'ME' instead.
  monthly_cases = covid_df.resample('M')['new_cases_today'].sum()
```




```
<ipython-input-16-b4f08fe1d526>:20: FutureWarning: DataFrame.fillna
with 'method' is deprecated and will raise in a future version. Use
obj.ffill() or obj.bfill() instead.
  ).fillna(method='ffill')
```


Evoluția cazurilor COVID absolute pe județe
Bucuresti
Cluj
Timis
Ilfov
Constanta
Iasi
Brasov
Prahova
Bihor
Sibiu
Dolj
Arges
Arad
Mures
Galati
county_name
reporting_date
Număr total de cazuri
600000
500000
400000
300000
200000
100000


Relația dintre testare și rata de pozitivare în timp (cu medie normală)
Teste zilnice
Medie teste: 13768
Rată de pozitivare zilnică (%)
Medie pozitivare: 24.07%
Prag OMS (5%)
Număr de teste
Rată de pozitivare (%)
40000
30000
20000
10000
0
250
200
150
100
50
0
2020-05
2020-09
2021-01
2021-05
2021-09
2022-01
2022-05
2022-09
2023-01


Distribuția geografică a cazurilor COVID în raport cu populația
Bucuresti (345.8)
Cluj (263.3)
Timis (244.7)
Ilfov (273.6)
Brasov (225.5)
Constanta (196.2)
Iasi (158.8)
Prahova (156.5)
Bihor (164.6)
Sibiu (223.7)
Dolj (134.1)
Arges (141.3)
Arad (188.0)
Bacau (119.3)
Suceava (104.8)
Hunedoara (175.4)
Alba (184.0)
Dambovita (120.2)
Neamt (116.8)
Valcea (131.9)
Olt (112.2)
Vaslui (114.0)
Botosani (106.5)
Giurgiu (121.7)
Salaj (144.4)
Ialomita (117.2)
Vrancea (93.9)
Calarasi (104.1)
Mehedinti (103.2)
Tulcea (117.5)
Caras-Severin (123.8)
Total cazuri COVID
Populație județ
Cazuri la 1000 locuitori
$10^5$
$10^6$
300
250
200
150
100

Am completat si feedback-ul

